Artificial Intelligence Review (2024) 57:242
https://doi.org/10.1007/s10462-024-10877-1

A comprehensive survey of deep learning-based lightweight object detection models for edge devices

Payal Mittal[1]

Accepted: 25 July 2024 / Published online: 10 August 2024
© The Author(s) 2024

Abstract
This study concentrates on deep learning-based lightweight object detection models on edge devices. Designing such lightweight object recognition models is more difficult than ever due to the growing demand for accurate, quick, and low-latency models for various edge devices. The most recent deep learning-based lightweight object detection methods are comprehensively described in this work. Information on the lightweight backbone architectures used by these object detectors has been listed. The training and inference processes concerning to deep learning applications on edge devices is being discussed. To raise readers' awareness of this developing domain, a variety of applications for deep learning-based lightweight object detectors and related utilities have been offered. Designing potent, lightweight object detectors based on deep learning has been suggested as a counter to such problems. On well-known datasets such as MS-COCO and PASCAL-VOC, we thoroughly examine the performance of certain conventional deep learning-based lightweight object detectors.

Keywords Deep learning · Lightweight networks · Object detection · Computer vision · Edge devices · Computing power

1 Introduction

The advancement of effective deep learning-based object detectors has been influenced by Internet of Things (IoT)-based technologies. The majority of deep object models demand too much Central Processing Unit (CPU) power and cannot be used on edge devices, despite the fact that many object detectors attain outstanding accuracy and carry out inference in real-time (Wang et al. 2021a, 2021b, 2021c, 2022). Exciting outcomes have already been achieved using a variety of strategies. The brief study of strategies to deployment of deep learning-based applications into edge devices include (Wang et al. 2020a, 2020b, 2020c, 2021a, 2021b, 2021c; Véstias et al. 2020; Li and Ye 2023; Subedi et al. 2021):

✉ Payal Mittal
payal@thapar.edu

[1] CSED, Thapar Institute of Engineering & Technology, Patiala, India

- Using a partitioning technique, since various layers may execute at different times. For example, in a fully connected or convolutional layer, divide the processing graph into offloadable tasks so that the execution time of each composite task unit is the same.
- Large-scale analytics platforms require intermediate resource standardisation for data manageability and low latency, as opposed to standalone applications on mobile devices. With the provisioning of intermediate resources, deep learning-based analytics platform can determine the proportion of local processing, provided that there is a mechanism to divide the load between buffering and memory loading. The offloaded execution through efficient partitioning can reduces costs, latency, or any other issue-related aim.

Moreover, a detailed study is provided in Sect. 4.6 of manuscript. In recent years, a new field of study i.e., lightweight object detectors have emerged with the goal of developing compact, effective networks for deployments of the IoT that frequently take place in low computing or resource-constrained settings. The research community has long worked to identify the best accuracy detection models through advanced architectural searches, as developing the deep learning-based lightweight network architecture is a difficult procedure. When using these models in edge devices, such as high-performance embedded processors, the question arises regarding usage of high-end innovative applications with fewer resources. It is still not entirely possible to perform detection using a smart phone or edge devices. Although existing models available today are capable of doing this task, but their precision level is just insufficient and undesirable in real-time instances.

Edge computing, according to Gartner, is a component of an architecture of distributed computing where data processing resides near the edge where devices or individuals generate or consume that data (Hua et al. 2023). Because of the constant growth in data created by the IoT, edge computing was first allocated to reduce bandwidth costs for data travelling long distances. On the other hand, the emergence of real-time applications that require processing at the edge is driving the current technological advancements. Among many other benefits, data minimization at the network level can prevent bottlenecks and significantly reduce energy, bandwidth, and storage expenses. A single device is able to send data across a network, problems occur when hundreds of devices send data at once. In addition to reducing quality due to delay, it also raises bandwidth expenses and creates bottlenecks that might result in cost spikes. By acting as a local source for these systems' data processing and storage, edge computing services and offerings assist in fixing this problem. It also serves as an edge gateway, minimizing bandwidth requirements by processing data from an edge device and sending the pertinent data back through the cloud (Jin et al. 2021). A key element in modern integrated real-world Artificial Intelligence (AI) systems is edge devices. IoT devices could only gather data in the beginning and send it to the cloud for processing. By putting services closer to a network's edge, edge computing expands the possibilities of cloud computing and enables a wider range of AI services and machine learning applications. IoT computing devices, mobile devices, embedded computers, smart TVs, and other connected gadgets can all be termed edge devices. Real-time application development and deployment can be accelerated by edge computing devices through high-speed networking technologies such 5G networking. Robotics, image and video processing, intelligent video analytics, self-driving cars, medical imaging, machine vision, industrial inspection, among examples of such applications (Véstias et al. 2020).

Edge computing can be applied to devices that are directly connected to sensors, routers or gateways that transfer data, or small servers installed locally in a closet. There are an increasing number of edge computing use cases as well as smart devices capable

of doing various activities at the edge. The range of applications for edge computing is expanding in tandem with the development of AI capabilities. The applications spanning a wide range can be found utilising edge computing (Xu et al. 2020). Additionally, there is a good deal of overlap among the various use cases for edge computing. In particular, edge computing functionality in traffic management systems is closely related to that of autonomous vehicles as briefly discussed below:

(a) Industrial infrastructure
Predictive maintenance and failure detection management in industries are supported by the edge computing. When a machine or component breaks down, the capability kicks in, enabling factory workers to fix the issue or replace the part in advance and save money by preventing lost output. The architecture of edge computing can handle large amounts of data from sensors and programmable logic controllers, as well as facilitate effective communications across extremely complicated supervisory control and data gathering systems.

(b) Retail
Huge amounts of data are produced by retail applications from different point-of-sale systems, item stocking procedures, and other company operations. Edge computing can assist in analysing this vast quantity of data and locating problems that require quick resolution. Additionally, edge computing provides a way to handle consumer data locally, preventing it from leaving the client's residence, a privacy regulation problem that is becoming more pressing.

(c) Healthcare
In order to give medical practitioners precise, timely information about a patient's status, the healthcare and medical industries gather patient data from sensors, monitors, wearable technology, and other devices. Edge computing solutions can provide dashboards with such data so users can see all the key indications in one convenient place. AI-enabled edge computing solutions can recognise anomalous data, allowing medical personnel to respond to patient requirements quickly and with the minimal possible false alarms. Furthermore, edge computing devices can aid in addressing concerns related to patient confidentiality and data privacy by processing data locally.

(d) Global energy
Cities and smart grid systems can monitor public buildings and facilities for improved energy efficiency in areas like lighting, heating, and clean energy use by using edge computing devices. As an illustration: edge computing devices are utilised by intelligent lighting controls to regulate individual lights for optimal efficiency and public space safety; Embedded edge computer devices are used in solar fields to detect changes in the weather and modify their position; Edge computing is used by wind farms to send sensor data to substations and link to cell towers.

(e) Public transit systems
Only the data necessary to support in-car activities and dispatcher insights in public transportation applications can be collected and transmitted by edge computing systems deployed in buses, passenger rail systems, and paratransit vehicles.

(f) Travel transport utilities
In order to increase convenience and safety, edge computing can control when traffic signals turn on and off, open and close additional lanes of traffic, make sure that communications are maintained in the event of a public emergency, and do other real-

time tasks. The adoption of autonomous vehicles will be significantly influenced by sophisticated traffic management systems, as was previously indicated.

(g) Advanced industries

In advanced industries, vehicle sensors and cameras can provide data to edge computing devices, which make choices in milliseconds without any latency. This fast decision making is necessary in autonomous vehicles, for safety reasons. Self-parking apps and lane-departure warning are two examples of edge computing services that are currently readily accessible. Furthermore, as more cars are able to communicate with their surroundings, a quick and responsive network will be required. In order to assist predictive maintenance, electric vehicles require constant monitoring. Edge computing can be used to manage data in this regard. Data aggregation is supported by edge computing, which reports actionable data for maintenance and performance. These above-mentioned multitude of industries investing in implacability of edge devices. These industries include travel, transport and logistics, cross-vertical, retail, public sector utilities, global energy and materials, banking insurance, infrastructure and agriculture etc. Their share representation with respect to employability in various edge computing devices is shown in Fig. 1a (Chabas et al. 2018). The travel, transport and logistics holds the maximum share of 24.2%, then 13.1% in global energy markets, 10.1% in retail and advanced industries followed by less shares by other industries. We have also represented hardware costs comparisons in terms of minimum and maximum cost in case of edge computing devices for mentioned industries. The hardware value includes opportunity across the tech stack on the basis of sensors, on-device firmware, storage and processor. By 2025, the edge computing-based devices depicts $175 to $215 billion potential hardware value. The industries such as travel, transport and logistics approximate $35 to $43, cross-vertical estimated to be $32 to $40 billion, $20 to $28 billion in retail sector, $16 to $24 billion in public sector utilities, $9 to $17 billion in global energy and materials, $4 to $11 billion in infrastructure and agriculture as depicted in Fig. 1b (Chabas et al. 2018). There is a dire need to focus on advancing development of lightweight object detection models to boost their employability in heterogeneous edge devices. This survey study analyses the state-of-the-art deep learning-based lightweight object identification models in order to attain excellent

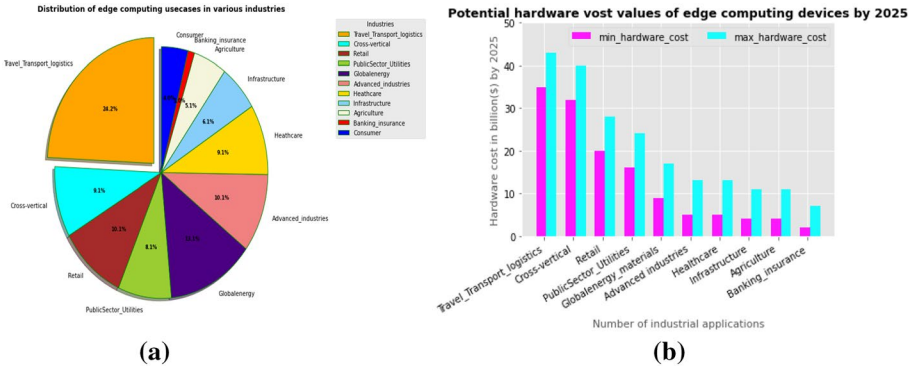

Fig. 1 **a** Share representation of various industries embedded in edge computing devices. **b** Comparison of hardware costs in case of edge computing devices

performance on edge devices. With equivalent accuracy, powerful lightweight object detection models offer these advantages (Kim et al. 2023; Huang et al. 2022):

(1) Lightweight object detection models based on deep learning require less communication between edge distributed training.
(2) Less bandwidth will be needed to export a cutting-edge detection model from the cloud to a particular application.
(3) Deploying lightweight detectors on Field Programmable Gate Arrays (FPGAs) and other hardware with limited resources is more practical.

1.1 Motivation

Object detection is the core concept in deploying innovative edge devices-based applications such as face detection (Li et al. 2015), objects tracking (Nguyen et al. 2019), video surveillance (Yang and Rothkrantz 2011), pedestrian detection (Brunetti et al. 2018), etc. The powerful capabilities of deep learning boost the performance of object detection in these applications. The generic deep learning-based object detection models have computational complexities such as extensive use of platform resources, more bandwidth, and large data processing pipelines (Jiao et al. 2019; Zhao et al. 2019). However, a detection network might potentially use three orders of magnitude more Floating Point Operations (FLOPs) than a classification network due to the computational complexity, making its deployment on an edge device much more difficult (Ren et al. 2018). The generic deep object detectors often use more network layers which eventually require high parameter tuning. Deep models have more network layers, which makes it harder for the network to detect small targets because they lose position and feature information over time. The network parameters being too large could damage the model's effectiveness and make it challenging to implement on smart mobile terminals, which brings us to our final possible concern.

For the development of lightweight object detection on edge devices, a comprehensive assessment of the research directions related to this topic is necessary, particularly for researchers who are interested in pursuing this line of inquiry. To assess the usefulness of deep learning-based lightweight object detection on edge devices, more research is required than just a basic review of the literature. Because the proposed research can offer a comprehensive examination of the literature, it can achieve each of these objectives. A deep learning-based lightweight detection evaluation hasn't been written about recently in the literature. There are generic and application specific surveys dedicated to deep learning-based object detectors (Jiao et al. 2019; Zou et al. 2023; Liu et al. 2020a, 2020b, 2020c, 2020d; Mittal et al. 2020; Han et al. 2018; Zhou et al. 2021a, 2021b) but not have consolidated study specifically for lightweight detectors for edge devices as mentioned in Table 1. To raise readers' understanding of this developing subject, deep learning-based lightweight object detectors on edge devices have been investigated in this work. The research of deep learning-based lightweight object identification models with regard to various backbone architectures and diverse applications on edge devices will be advanced by the release of this study. The key objectives of the survey are as follows:

Table 1 Comparison of existing object detection related publications with proposed work

Studies	Description	Year	Deep learning-based studies	Lightweight object detection	Edge devices	Backbone architectures	Multiple applications
Agarwal et al. (2018)	Highlighted object detection based region proposal generation	2016	√	×	×	×	√
Han et al. (2018)	Discussed salient and category-specific detection	2018	√	×	×	×	×
Zhao et al. (2019)	Focused on generic object detection methods	2019	√	×	×	√	×
Zou et al. (2023)	Provide details of object detectors, and datasets	2019	√	×	×	×	×
Liu et al. (2020a, 2020b, 2020c, 2020d)	Introduced aspects of generic object detection	2020	√	×	×	√	×
Wu et al. (2020)	Discussed recent advances in object detection	2020	√	×	×	√	√
Sharma and Mir (2020)	Systematic study of deep object detection techniques	2020	√	×	×	×	√
Xiao et al. (2020)	Thorough review of object detection technology	2020	√	×	×	√	√
Rodriguez-Conde et al. (2021)	Presented object detection for privacy-compliant visual perception	2021	√	√	√	×	√
Liu et al. (2021a, 2021b, 2021c)	Provide a comprehensive study for small object detection	2021	√	×	×	×	×
Zaidi et al. (2022)	Discussed modern deep object detection models	2022	√	√	×	√	×
Kamath and Renuka (2023)	Study on resource-constrained object detection	2023	√	√	√	×	√
	Survey of lightweight object detection for edge devices	2023	√	√	√	√	√

- To provide taxonomy of deep learning-based lightweight object detection algorithms on edge devices
- To provide an analysis of deep learning-based lightweight backbone architectures for edge devices
- Literature findings of applications deployed through lightweight object detectors
- Comparison of lightweight object detectors by analyzing results on leading detection datasets

The organization of research paper is as follows: Sect. 2 elaborates the work related to development of deep learning-based object detectors. The deep learning-based object detectors have further categorized into two, one and advanced stage. Section 3 describes materials and methods required for deep learning-based lightweight detection models on edge devices. The architectural details related to training and inference lightweight models have also been mentioned in this section. Further, detailed crucial properties and performance milestone of lightweight object detection methods have been mentioned in this section. Section 4 discusses commonly utilized backbone architectures in deep learning-based lightweight object detection models. Further, applications of lightweight object detection models have also been mentioned. The recommendations for designing powerful deep learning-based lightweight model are provided in Sect. 4. The final section brings the entire study to a close and outlines some crucial implications for more research.

2 Background

Recent developments in the field of deep learning-based object detectors have mostly concentrated on raising the benchmark datasets' state-of-the-art accuracy, which has caused an explosion in model size and parameters. The research, on the other hand, has demonstrated interest in suggesting lighter, smaller, and smarter networks that would minimise the parameters while keeping cutting-edge performance (Nguyen et al. 2023). In the next section, we will provide a brief summary regarding categorization of generic object detection models.

2.1 Taxonomy of deep learning-based object detectors

During the last years, there has been a rapid and successful expansion in lightweight object detection research domain. This domain has exploded from adopting and familiarizing the latest machine and deep methods through development of new representations. The generic deep learning-based object detection models have been classified into two, one and, advanced stage each having different concepts.

2.1.1 Two-stage object detection models

Two-stage algorithms, having two different stages of region proposal and detection head. The first stage was for the calculation of RoI proposals using anchors in external region proposal techniques such as Edge Box (Zitnick and Dollár 2014) or Selective Search (Uijlings et al. 2013). The second stage consists of processing extracted RoIs into final bounding boxes, coordinate values and class labels. The examples of two-stage algorithms

include Faster RCNN (Ren et al. 2015), Cascade RCNN (Cai and Vasconcelos 2018), R-FCN (Dai et. al. 2016) etc. The advantages of two-stage object detectors include better analysis of objects through given stages, multi-stage architecture to regress the bounding box values efficiently and better handling of class imbalance in datasets. Two-stage detectors adopted a deep neural-based Region Proposal Network (RPN) and a detection head. Even if the existing Light-Head R-CNN (Li et al. 2017) used a lightweight detection head, the backbone and detection part become imbalanced when the detection part is combined with a small backbone. This mismatch increases the danger of overfitting and causes repetitive calculation.

2.1.2 One-stage object detection models

Two-stage detectors helped deep learning-based object detection get off to a good start, but these systems struggled with speed. Due to their flexibility in satisfying demands like fast speed and minimal memory needs, one-stage detectors were ultimately adopted by researchers. The region proposal stage of two-stage detectors was eliminated by the one-stage algorithms since they saw the object identification problem as a regression problem. Instead of sending portions of the image to a fixed grid-based CNN, the entire image is sent at once, and anchors assist in identifying specific region suggestions. For the purpose of detecting the given area in a picture, boundary box coordinates were included. The examples of one-stage detectors include YOLO (Redmon et al. 2016), SSD (Liu et al. 2016), RetinaNet (Lin et al. 2017a, 2017b) etc. The YOLO series outperforms two-stage models in terms of efficiency and accuracy.

2.1.3 Advanced-stage object detection models

The recently emerged advanced-stage object detectors removed the anchors concept in one-stage detectors for detecting objects. The advanced detector, CornerNet (Law and Deng 2018) detected objects as paired key points and a new corner pooling layer was introduced to better localize corners. CenterNet (Duan et al. 2019) detected the object as a triplet, rather than a pair of key points. Foveabox (Kong et al. 2020a, 2020b) predicted category-sensitive semantic maps and category-agnostic bounding box for the object. The advanced-stage detectors also found struggling in locating multiple targets having small-size, complex backgrounds and slow detection speed. The one-stage methods (Bochkovskiy et al. 2020; Qin et al. 2019) utilized predefined anchor boxes and anchor-free (Duan et al. 2019) concepts for predicting bounding boxes.

2.1.4 Light-weight object detection models

The low computation in terms of bandwidth and resource utilization are light-weight object detectors and few examples include ThunderNet (Qin et al. 2019), PP-YOLO (Long et al. 2020a, 2020b), YOLObile (Cai et al. 2021), Trident-YOLO (Wang et al. 2022a, 2022b, 2022c, 2022d), YOLOV4-tiny (Jiang et al. 2020), Trident FPN (Picron and Tuytelaars 2021) etc.

The deep learning-based object detection algorithms have been categorized into two, one, advanced-stage and light weight detectors are highlighted in Fig. 2. The algorithms such as Faster RCNN (Ren et al. 2015), Mask RCNN (He et al. 2017), Cascade RCNN (Cai and Vasconcelos 2018), FPN (Lin et al. 2017a, 2017b) and R-FCN (Dai et al. 2016)

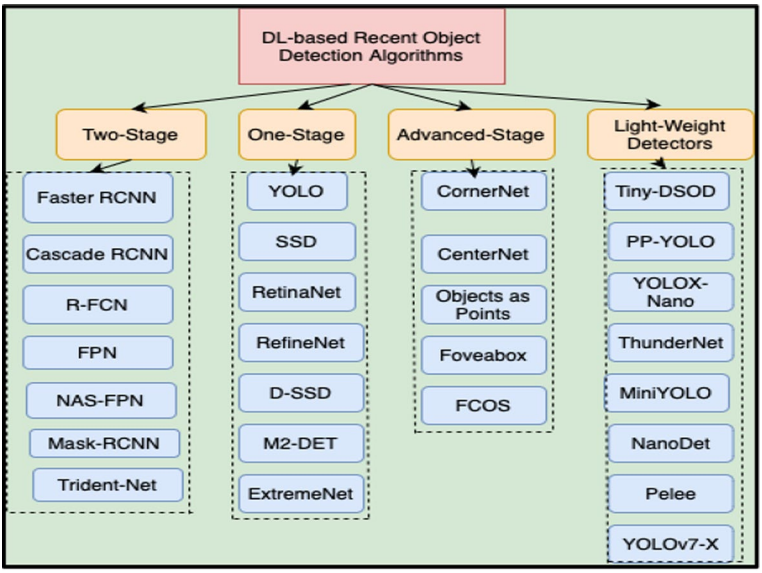

Fig. 2 Taxonomy of recent deep learning-based object detection algorithms

etc., fall under two-stage detectors whereas YOLO (Redmon and Farhadi 2018), SSD (Liu et al. 2016), RefineDet (Zhang et al. 2018a, 2018b) and RetinaNet (Lin et al. 2017a, 2017b) under one-stage detectors. The advanced object detectors such as CornerNet (Law and Deng 2018), Objects as points (Zhou et al. 2019a) and Foveabox (Kong et al. 2020a, 2020b) are listed in Fig. 2. However, the algorithms listed above often include a large number of channels and convolutional layers, which demand a lot of computing power for deployment in edge devices. The deep learning-based lightweight object detectors presented in Fig. 2 are specifically designed for contexts with limited resources. Due to their efficiency and compactness, the one and advanced stage detectors' pipeline is the industry standard for designing lightweight object detectors.

3 Deep learning-based lightweight object detection models for edge devices

Numerous computer vision tasks, such as autonomous driving, robot vision, intelligent transportation, industrial quality inspection, object tracking, etc., have used deep learning-based object detection to a large extent. Deep models typically improve performance, but the deployment of real-world applications onto edge devices is constrained by their resource-intensive network. Lightweight mobile object detectors have drawn growing research interest as a solution to this issue, with the goal of creating extremely effective object detection. Deep learning-based lightweight object detectors have recently been developed for situations with limited computer resources, such as mobile devices.

The necessity to execute backbone designs on edge devices with constrained memory and processing power stimulates research and development of deep learning-based lightweight object identification models. A number of efficient lightweight backbone architectures have been proposed in recent years, for example, MobileNet (Howard et al. 2017),

ShuffleNet (Zhang et al. 2018a, 2018b), and DetNaS (Chen et al. 2019). However, all these architectures are heavily dependent on widely deployed depth-wise separable convolution-based methodologies (Ding et al. 2019). With regard to deep learning-based lightweight object identification models, we will describe methodology and each component in depth in the following sections. Our deep learning-based simple object detection models were heavily influenced by existing simple and complex object detection models. We give an architectural breakdown of deep learning-based lightweight object detection models in the following section.

3.1 Architecture methodology of lightweight object detection models

The different building blocks of deep learning-based lightweight object detection algorithms on edge devices consist of number of components consisting of input, backbone, neck and detector head. The definition and details of each component is tabulated in Table 2. An input for the lightweight object detector is either an image, patch or pyramid, initially fed in the lightweight backbone architecture such as CSPDarkNet (Redmon and Farhadi 2018), ShuffleNet (Zhang et al. 2018a, 2018b), MobileNet (Qian et al. 2021), PeleeNet (Wang et al. 2018) for the calculation of feature maps. The backbone is the part of deep learning-based lightweight object detection architecture which converts an image to feature maps, whereas the neck transforms the feature maps by connecting the backbone to detector head. The input image is passed to lightweight backbone architecture to calculate initial features vectors of objects. This backbone network may be a pre-trained network or a neural network built from scratch with the aim of feature extraction. The backbone architecture performs feature extraction and produces feature maps as an output. Then, neck component transforms this feature map to a required feature vector for handling various object detection challenges as per application. The lightweight detector head can be visualized as a deep neural network focusing on extraction of RoIs. Further, some pooling layer fixes the size of calculated RoIs to calculate final features of the detected objects. The final features are then passed onto classification and regression loss functions to assign class labels and regressing the coordinates values of bounding boxes. This whole process is repeated until the final regressed values of bounding boxes are obtained with the required class labels. The detailed methodology as presented in Fig. 3, deep learning-based lightweight object detection consist of three parts i.e., backbone architecture, neck components, and lightweight head prediction. The input images are fed to the backbone and their architecture converts the input image into feature maps. In case of deep learning-based lightweight models, the backbone architecture should be deployed from given categories in Table 2. The Conv2D + batch normalization + ReLU activation function is represented by a fundamental convolutional module that makes up the backbone architecture. By eliminating redundant gradient information from CNN's optimization process and integrating gradient modifications into the feature map, it lowers input parameters and model size (Wang et al. 2020a, 2020b, 2020c). In the bottleneck cross stage partial darknet model, for instance, a 640 × 640-pixel image is divided into four 320 × 320-pixel images, which are then combined to form a 320 × 320-pixel feature map. This 320 × 320x32 resulting feature map was produced using 32 convolutional kernels. Additionally, include the SPP module to add features of various sizes and increase the network's receptive area. By enhancing the information flow between the backbone architecture and the detecting head, the neck alters the feature maps. The neck, PANet is built on a FPN topology utilized to provide strong

Table 2 Building blocks of deep learning-based lightweight object detectors

Input	Image, patches, image pyramids	An input is passed to the backbone architecture
Backbone	CSPDarkNet (Redmon and Farhadi 2018), ShuffleNet Zhang et al. (2018a, 2018b), MobileNet (Sandler et al. 2018), mNASNet (Tan et al. 2019), MobileViT (Mehta and Rastegari 2021)	A backbone is a pre-trained network on a standard dataset like ImageNet used for feature extraction
Neck	FPN (Lin et al. 2017a, 2017b), PANet (Wang et al. 2019), Bi-FPN (Quang et al. 2021)	Placed between backbone and head to transform feature maps
Lightweight detector head	Tiny YOLOv4 (Jiang et al. 2020), NanoDet (Li et al. 2020a, 2020b), Scaled YOLOv4 (Bochkovskiy et al. 2020), PP-PicoDet (Yu et al. 2021)	A detector head is used to predict classes and bounding boxes of objects

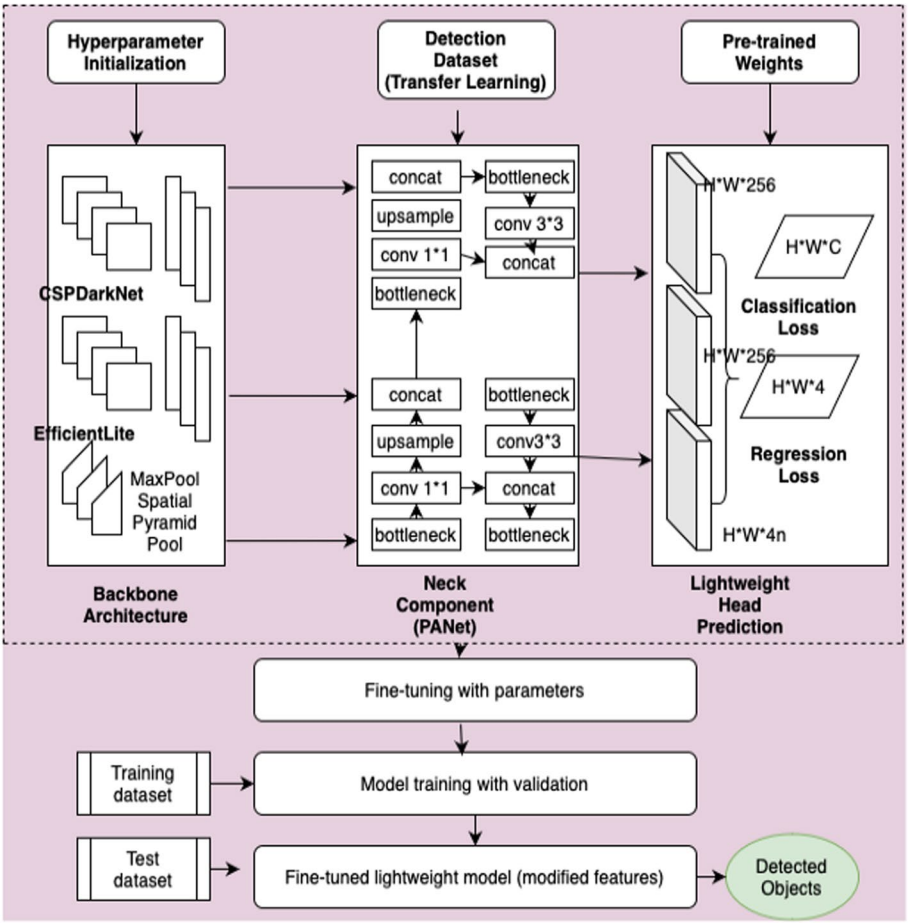

Fig. 3 Methodology of deep learning-based lightweight object detection model

semantic characteristics from top to bottom (Wang et al. 2019). FPN layers from bottom to top also express important positional features.

Furthermore, PANet encourages the transmission of low-level characteristics and the use of precise localization signals in the bottom layers. This improves the target object's position accuracy. The prediction layer, sometimes referred to as the detection layer, creates many feature maps in order to accomplish multiscale prediction. However, at the prediction layer, the model is capable of classifying and detecting objects of various sizes. As a result, it is projected that each feature map will have various regression bounding boxes at each position, yielding various regression bounding boxes. The anticipated output of the model with bounding boxes is then shown as a detection result. The three steps mentioned above combine the training model for detection into the lightweight object detection model. After model training, the test data is passed to get fine-tuned lightweight model with modified features as shown in Fig. 3. The parameters in context of deep learning-based light-weight models are discussed below:

(a) Training

To train an edge-cloud-based deep learning model, edge devices and cloud servers must share model parameters and other data. More data must be transferred between edge devices and cloud servers as the training model gets bigger. A number of methods have been put forth to lower the cost of communication during training, including Edge Stochastic Gradient Descent (eSGD), which can reduce a CNN model's gradient size by up to 90% by communicating only the most important gradients, and intermediate edge aggregation prior to federated learning server aggregation. The two main components of training deep learning-based lightweight detection models are the ability to exit before the input data completes a full forward pass through each layer of a neural network distributed over heterogeneous nodes and the use of binarized neural networks to reduce memory and compute load on resource-constrained end devices (Koubaa et al. 2021; Dey and Mukherjee 2018).

Researchers have created a novel architecture known as Agile Condor that carries out real-time computer vision tasks using machine learning methods. At the network edge, close to the data sources, Agile Condor can be utilised for autonomous target detection (Isereau et al. 2017). Precog is a new method that lowers latency for mobile applications by prefetching and caching that anticipates the subsequent classification request and uses end-device caching to store essential portions of a trained classifier. As a result, fewer offloads to the cloud occur and edge servers calculate the likelihood that linked end devices may make a request in the future. These pre-fetched modules function as smaller models that minimise network traffic and cloud processing while accelerating inference on the end devices (Drolia et al. 2017). Another example include ECHO is a feature-rich, thoroughly tested framework for implementing data analytics in a distributed hybrid Edge-Fog-Cloud configuration. ECHO offers services such virtualized application status monitoring, resource discovery, deployment, and interfaces to data analytics runtime engines (Ogden and Guo 2019).

(b) Inference

When feasible, distributed deep network designs enable the deployment on edge-cloud infrastructure to support local inference on edge devices. A distributed neural network model's ability to function effectively on minimising inter-device communication costs. Inference on the end-edge-cloud architecture is a dynamic problem because of evolving network conditions (Subedi et al. 2021). Static methods like remote inference only or on-device inference only are also not the best. Ogden and Guo have created a distributed architecture that provides a flexible answer to this problem for mobile deep inference. A centralised model manager will house many deep learning models, and the inference environment (memory, bandwidth, and power) will be used to dynamically determine which model should run on which device. If resources are scarce in the inference environment, one of the compressed models may be employed; if not, an uncompressed model with higher accuracy is used. Edge servers handle remote inference when networks are sluggish.

(c) Privacy and security

Edge devices can be used to filter personally identifiable information prior to data transfer in order to enhance user privacy and security when processing data remotely (Xu et al. 2020; Hu et al. 2023a, 2023b). Since data generated by end devices is not available to a central location, training deep learning models across several edge devices in a distributed way leads to more privacy. Personally identifiable information in photographs and videos can be removed at the edge before being uploaded to an external server, enhancing user privacy. The privacy of critical training data becomes

an issue when training is conducted remotely. To ensure local and global privacy techniques, it is imperative to keep an eye out for any decline in accuracy, ensure low computing overheads, and provide resilience to communication errors and delays (Abou et al. 2023; Makkar et al. 2021).

3.2 Comprehensive analysis of lightweight object detection models

The development of extremely effective object detection outcomes has garnered increasing scientific attention in the small, transportable object detectors. With the use of efficient components and compression techniques like pruning, quantization, hashing, and other techniques, the effectiveness of deep learning lightweight object identification models has grown. Distillation, which employs a large network that has been used to train smaller models, has produced some surprising results as well. A comprehensive list containing multiple details of deep learning-based lightweight object detection models in the recent years is presented in Tables 3, 4. The categorization of anchor-based and anchor-free detectors for lightweight object detectors have been identified. Anchor-based methods are the mechanism of extracting RoIs employed in object detection models, such as Fast R-CNN (Girshick 2015). The anchor boxes are of various scales, which can be viewed as RoIs, as a priori for performing bounding box regression for coordinates values. The detectors including YOLOv2 (Redmon and Farhadi 2017), YOLOv3 (Redmon and Farhadi 2018), YOLOv4 (Bochkovskiy et al. 2020), RetinaNet (Lin et al. 2017a, 2017b), RefineDet (Zhang et al. 2018a, 2018b), EfficientDet (Tan et al. 2020), Faster R-CNN (Ren et al. 2015), Cascade R-CNN (Cai and Vasconcelos 2018), Trident-Net (Li et al. 2019), belonging to one and two-stage detectors have anchor mechanism to elevate the performance of deep learning-based object detection. Besides, anchor-free detectors have recently received more attention in academia and research by witnessing a large number of new anchor-free methods have been proposed. Earlier works such as YOLOv1 (Redmon et al. 2016), DenseBox (Huang et al. 2015) and UnitBox (Yu et al. 2016) can be considered as early anchor-free detectors. In anchor-free methods, anchor and key points are utilized to perform detection. The former approach does object bounding box regression-based on anchor points instead of anchor boxes, including FCOS (Detector 2022), FoveaBox (Kong et al. 2020a, 2020b), whereas latter approach reformulates the object detection as keypoints localization problem, including CornerNet (Law and Deng 2018; Law et al. 2019), CenterNet (Duan et al. 2019), ExtremeNet (Zhou et al. 2019b) and RepPoint (Yang et al. 2019). By eliminating the handcraft anchors' restrictions, anchor-free techniques have a lot of promise for working with extremely large and small objects. The anchor-based detectors shown in Table 3 can compete with some newly proposed anchor-free lightweight object detectors in terms of performance. Further, input image type, code link and published sources are also mentioned in Table 3. While Table 4 reports crucial milestones such as AP, description, loss function etc. for individual deep learning-based light-weight detector.

Tiny-DSOD (Li et al. 2018) a lightweight object detector inspired by a thoroughly supervised object detection framework, has been proposed for resource-constrained applications. With only 0.95 M parameters and 1.06B FLOPs, it uses depth-wise dense block as a backbone architecture and depth-wise FPN in neck components, which is by far the most advanced result with such a small resource demand. The context enhancement module and the spatial attention module of ThunderNet (Qin et al. 2019), a lightweight two-stage detector, are used as the backbone architectural blocks to produce more discriminative feature representation representation. The effective RPN used in a

Table 3 A comprehensive list containing details of deep learning-based lightweight object detection models

S. No.	Authors	Detector	Type	Input image	Published in	URL link
1	Li et al. (2017)	YOLOv3-DarkNet53	Anchor-based	320*320	arXiv 2018	https://github.com/westerndigitalcorporation/YOLOv3-in-PyTorch
2	Howard et al. (2017)	MobileNet-SSD	Anchor-based	300*300	arXiv 2017	https://github.com/chuanqi305/MobileNet-SSD
3	Sandler et al. (2018)	MobileNetv2-SSDLite	Anchor-based	320*320	CVPR 2018	https://github.com/tranleanh/mobilenets-ssd-pytorch
4	Li et al. (2018)	Tiny-DSOD	Anchor-based	300*300	arXiv 2018	https://github.com/lyxok1/Tiny-DSOD
5	Wang et al. (2018)	Pelee	Anchor-based	304*304	NeurIPS 2018	https://github.com/Robert-JunWang/Pelee
6	Qin et al. (2019), Huang et al. (2018)	YOLO-LITE	Anchor-based	416*416	ICBD 2018	https://github.com/reu2018DL/YOLO-LITE
7	Qin et al. (2019)	ThunderNet	Anchor-based	320*320	ICCV 2019	https://github.com/DayBreak-u/Thundernet_Pytorch
8	Tan et al. (2019)	MnasNet-A1 + SSDLite	Anchor-based	320*320	CVPR 2019	https://github.com/tensorflow/tpu/tree/master/models/official/mnasnet
9	Tang et al. (2020a, 2020b)	LightDet	Anchor-based	320*320	ICASSP 2020	Not available
10	Yi et al. (2019)	YOLOV3-Tiny	Anchor-based	416*416	ICACCS 2020	https://github.com/pjreddie/darknet/blob/master/cfg/yolov3-tiny.cfg
11	Long et al. (2020a, 2020b)	PP-YOLO	Anchor-based	608*608	CVPR 2020	https://github.com/PaddlePaddle/PaddleDetection
12	Long et al. (2020a, 2020b)	YOLOv4-Tiny	Anchor-based	416*416	arXiv 2020	https://github.com/truong2710-cyber/Mask-Detection-YOLOv4-tiny-Kaggle-Dataset
13	Tan et al. (2020)	EfficientDet	Anchor-based	512*512	CVPR 2020	https://github.com/xuannianz/EfficientDet
14	Huang et al. (2021)	PP-YOLOv2	Anchor-based	640*640	arXiv 2021	https://github.com/PaddlePaddle/PaddleDetection
15	Ge et al. (2021)	YOLOX-Nano	Anchor-free	416*416	ICSP 2022	https://github.com/Megvii-BaseDetection/YOLOX
16	Ge et al. (2021)	YOLOX-Tiny	Anchor-free	416*416	IJCINI 2022	https://github.com/TexasInstruments/edgeai-yolox/blob/main/exps/default/yolox_tiny.py
17	Cai et al. (2021)	YOLObile	Anchor-based	320*320	AAAI 2021	https://github.com/nightsnack/YOLObile

Table 3 (continued)

S. No.	Authors	Detector	Type	Input image	Published in	URL link
18	Wang et al. (2021)	Scaled YOLOv4	Anchor-based	608*608	CVPR 2021	https://github.com/WongKinYiu/ScaledYOLOv4
19	Wang et al. (2021)	Trident YOLO	Anchor-based	416*416	Wiley IET	Not available
20	Li et al. (2021a, 2021b, 2021c)	NanoDet	Anchor-free	320*320	Journals of Radar	https://github.com/RangiLyu/nanodet
21	Yu et al. (2021)	PP-PicoDet	Anchor-free	416*416	arXiv 2021	https://github.com/PaddlePaddle/PaddleDetection
22	Ding et al. (2022)	Slim YOLOv4	Anchor-free	416*416	JRIP	Not available
23	Liu et al. (2022a, 2022b)	Mini YOLO	Anchor-free	320*320	Wiley Journal	Not available
24	Xu et al. (2022)	PP-YOLOE-S	Anchor-free	640*640	arXiv 2022	https://github.com/PaddlePaddle/PaddleDetection
25	Wang et al. (2022a, 2022b, 2022c, 2022d)	YOLOv7-X	Anchor-free	640*640	arXiv 2022	https://github.com/WongKinYiu/yolov7
26	Li et al. (2022a, 2022b)	L-DETR	Anchor-free	800*1333	IEEE Access	https://github.com/wangjian123799/L-DETR.git

Table 4 A list of crucial properties and performance milestone of lightweight object detection methods

S. No.	Detector	Backbone	Loss function	AP	Proposal
1	YOLOv3-DarkNet53	DarkNet53	Logistic regression	38.1	Network structure makes greater use of the GPU, making it faster to evaluate than Darknet-19
2	MobileNet-SSD	MobileNet	Smooth L1 loss	19.3	Lightweight deep network is constructed using depth-wise separable convolutions
3	MobileNetv2-SSDLite	MobileNetv2	Smooth L1 loss	22.1	With fewer parameters and less computational complexity, gets competitive accuracy
4	Tiny-DSOD	DDB-Net	Smooth L1 loss	23.2	For resource-constrained uses based on DDB and D-FPN blocks
5	Pelee	PeleeNet	Smooth L1 loss	22.4	Variant of DenseNet, built with conventional convolution
6	YOLO-LITE	Darknet-53	L1 loss	12.2	A real-time detection model developed to run on portable devices lacking a GPU
7	ThunderNet	SNet535	Sigmoid	28.1	Embedded context enhancement and spatial attention module
8	MnasNet-A1 + SSDLite	MnasNet-A1	Smooth L1 loss	23.0	Directly measures real-world inference latency by executing the model on edge devices
9	LightDet	Modified ShuffleV2	Smooth L1 loss	24.0	Introduce an efficient feature-preserving and refinement module
10	YOLOv3-Tiny	Reduced Darknet-53	Logistic regression	16.6	Lightweight model of YOLOv3, which takes reduced training time
11	PP-YOLO	MobileNetV3	IoU aware loss	45.9	Balanced efficiency, directly applied in real application scenarios
12	YOLOv4-Tiny	CSP-ResNet	CIoU loss	28.7	Reduced parameters, makes it suitable for edge devices
13	EfficientDet	EfficientNet	Focal loss	34.6	Proposed a weighted bi-directional FPN for feature fusion
14	PP-YOLOv2	ResNet101	IoU aware loss	49.5	Increasing the input size and follow the design of PAN to aggregate the top-down information
15	YOLOX-Nano	DarkNet53	GIoU loss	25.8	Dynamic label assignment strategy SimOTA
16	YOLOX-Tiny	CBAM	GIoU loss	32.8	Fuses convolutional attention and mixup data enhancement strategy
17	YOLObile	CSP-DarkNet53	GIoU loss	31.6	Offers mobile acceleration and block-punched pruning with a mobile GPU-CPU collaborative strategy
18	Scaled YOLOv4	CSPNet-15	CIoU loss	28.7	Propose a network scaling approach that modifies the width, and resolution of network
19	TridentYOLO	CSP-RFBs	Focal loss	40.3	Propose a trident feature pyramid network
20	NanoDet	ShuffleNetV2	Focal loss	20.6	Based on visual saliency and perform feature learning on samples added with saliency maps
21	PP-PicoDet	Enhanced ShuffleNet	GIoU Loss	30.6	Improved detection One-Shot NAS pipeline
22	Slim YOLOv4	MobileNetV2	CIoU loss	29.2	Efficient network computing methods for convolution

Table 4 (continued)

S. No.	Detector	Backbone	Loss function	AP	Proposal
23	MiniYOLO	DSLightNet	CIoU	21.4	Adopted depthwise separable convolution
24	PP-YOLOE-S	ResNet50-vd	IoU aware loss	43.1	Scalable backbone-neck architecture, and refined loss function
25	YOLOv7-X	RepCSPResNet	Assistant loss	53.1	Propose the trainable bag-of-freebies method to enhance accuracy
26	L-DETR	PP-LCNet	H-sigmoid function	–	Embedded group normalization in encoder-and-decoder module

portable detecting head. ThunderNet outperforms earlier lightweight one-stage detectors by operating at 24.1 frames per second with 19.2 AP on COCO on an ARM-based smartphone. One of the most recent, cutting-edge lightweight object detection algorithms, PP-YOLO (Long et al. 2020a, 2020b) employs MobileNetV3 (Qian et al. 2021), a practical backbone architecture for edge devices. The depth-wise separable convolutions used by PPYOLOtiny's detection head make it better suited for mobile devices. PPYOLOtiny adopts the optimisation techniques used by PPYOLO algorithms but does away with techniques that have a big impact on model size and performance. Block-punched pruning and a mobile acceleration unit with a mobile GPU-CPU collaboration approach are provided by YOLObile (Cai et al. 2021). Trident-YOLO (Wang et al. 2022a, 2022b, 2022c, 2022d) is an upgrade to YOLOV4-tiny (Jiang et al. 2020), designed for mobile devices with limited computing power. In neck components, Trident FPN (Picron and Tuytelaars 2021) improves the recall and accuracy of basic object recognition methods by reorganising the network topology of neck components. Trident-YOLO proposes fewer cross-stage partial RFBs and smaller cross-stage partial SPPs, as well as enlarging the receptive field of the network with the fewest FLOPs. Conversely, Trident-FPN significantly enhances lightweight object detection performance by increasing the computational complexity through an increase in a limited number of FLOPs and producing a multi-scale model feature map. In order to simplify computation, YOLOV4-tiny (Jiang et al. 2020) uses two ResBlock-D modules in place of two CSPBlock modules in the ResNet-D network. In order to extract more feature information about the object, such as global features, channel, and spatial attention, it also creates an auxiliary residual network block with consecutive 3×3 convolutions that is utilized to obtain 5×5 receptive fields with the goal of reducing detection error. Optimizing the original YOLOv4 (Bochkovskiy et al. 2020), Slim YOLOv4 (Ding et al. 2022) changes the backbone architecture from CSPDarknet53 to MobileNetv2 (Sandler et al. 2018). Separable convolution and depth-wise over-parameterized convolutional layers were chosen to minimize computation and enhance the performance of the detection network. Based on YOLOv2 (Redmon and Farhadi 2017; Wang et al. 2022a), YOLO-LITE (Huang et al. 2018; Wang et al. 2021a) offers a quicker, more effective lightweight variant for mobile devices. On a PC without a GPU, YOLO-LITE works at roughly 21 frames per second and 10 frames per second when used on a website with only 7 layers and 482 million FLOPS. Object recognition using Fully Convolutional One-Stage (FCOS) (Detector 2022) addresses the issue of label overlap within the ground-truth data. Unlike previous anchor-free detectors, there is no complex hyper-parameter adjustment. Large-scale server detectors constitute the majority of anchor-free detectors in general. The two small minority of anchor-free mobile device detectors are NanoDet (Li et al. 2020a, 2020b) and YOLOX-Nano (Ge et al. 2021). The issue is that compact anchor-free detectors typically struggle to strike a good balance between efficiency and accuracy. In order to choose positive and negative samples, the FCOS method NanoDet employs Adaptive Training Sample Selection (ATSS) (Zhang et al. 2020a, 2020b, 2020c) and uses generalised focal loss as the loss function for classification and bounding box regression. The centerness branch of FCOS and numerous convolutions on this branch are eliminated by the application of this loss function, which lowers the computational cost of the detection head. A lightweight detector dubbed L-DETR (Li et al. 2022a) is created based on DETR and PP-LCNet to balance efficiency and accuracy. L-DETR has fewer parameters with the new backbone than the DETR. It is utilised to compute the overall data and arrive at the final prediction. Its normalisation and FFN are enhanced, and thus raises the precision of frame detection. In Table 5, some well-known metrics in

Table 5 Performance parameters of deep learning-based lightweight object detectors

S. No.	Light-weight object detector	Backbone	FLOPs	Inference time (ms)	FPS	Parameters (M)	Real-time applications
1	YOLOv3-DarkNet53	DarkNet53	1453B	22	171	–	√
2	MobileNet-SSD	MobileNet	1.2G	–	59.3	4.31	*
3	MobileNetv2-SSDLite	MobileNetv2	0.8G	–	–	3.38	*
4	Tiny-DSOD	DDB-Net	1.12G	–	105	0.95	*
5	Pelee	PeleeNet	1.21B	–	205	5.98	*
6	YOLO-LITE	Darknet-53	0.48G	–	21	–	*
7	ThunderNet	SNet535	0.47	–	248	–	√
8	MnasNet-A1 + SSDLite	MnasNet-A1	0.8B	203	–	4.9	*
9	LightDet	Modified ShuffleV2	0.50G	–	250	–	√
10	YOLOv3-Tiny	Reduced Darknet-53	5.56 B	4.5	368	8.86	*
11	PP-YOLO	MobileNetV3	1.02G	10.48	73	1.08	√
12	YOLOv4-Tiny	CSP-ResNet	–	–	371	6.06	√
13	EfficientDet	EfficientNet	2.5B	10.20	98	3.9	√
14	PP-YOLOv2	ResNet101	0.115G	14.50	68.9	1.08	√
15	YOLOX-Nano	DarkNet53	1.08G	19.23	90.1	0.91	√
16	YOLOX-Tiny	CBAM	6.48G	32.77	–	5.06	√
17	YOLObile	CSP-DarkNet53	3.95G	–	17	4.59	√
18	Scaled YOLOv4	CSPNet-15	6.3B	–	62	53	√
19	TridentYOLO	CSP-RFBs	5.19B	–	29.3	–	√
20	NanoDet	ShuffleNetV2	1.2G	13.35	–	0.95	*
21	PP-PicoDet	Enhanced ShuffleNet	0.73G	8.13	–	0.99	√
22	Slim YOLOv4	MobileNetV2	–	–	60.19	–	√
23	MiniYOLO	DSLightNet	–	–	–	2.06	*
24	PP-YOLOE-S	ResNet50-vd	110.7G	12.8	208.3	52.20	√
25	YOLOv7-X	RepCSPResNet	189.9G	–	114	71.3	√
26	L-DETR	PP-LCNet	–	–	–	–	√

calculating performance of lightweight object detection models have been highlighted. The metrics termed FLOPs are frequently used to determine how computationally complex deep learning models are. They provide a quick and simple method of figuring out how many arithmetic operations are needed to complete a particular computation. It can offer extremely helpful insights on computational costs or requirements or energy consumption, which is particularly helpful for edge computing. It is useful when we have to estimate the total number of arithmetic operations needed, which is usually when computing efficiency is being measured. As highlighted, YOLOv7-x has highest FLOPs i.e., 189.9G among the mentioned detectors. One of the more important components of using a deep network architecture in deployment is the network latency/inference time. The majority of real-world applications need inference times that are quick—a few milliseconds to a second. It needs in-depth knowledge to measure a neural network's inference time accurately. The time it takes for a deep learning algorithm to process fresh input and produce a prediction is known as the inference time in deep learning. The number of layers, the complexity of the network, and the number of neurons in each layer can all impact this time. Inference times typically rise with network complexity

and scale. In our analysis, YOLOv3-Tiny has lowest inference time of 4.5 ms. The Frame Per Second (FPS) is a measure of how rapidly a deep learning model can handle frames. It also specifies how quickly your object detection model will process your photos and videos and produce the desired results. YOLOv4-Tiny has highest FPS among presented ones in Table 5. Weight and bias are the model parameters in deep learning, which are characteristics of the training data that will be discovered throughout the learning process. The total number of parameters, which is a common indicator of a model's performance, is the sum of all the weights and biases on the neural network. YOLO-X Nano has least learning parameters when compared with others. Moreover, with respect to each light-weight object detector, prediction regarding deployment of individual detector in real-time applications has been done on the basis of their AP values highlighted in Table 4. MobileNet-SSD, MobileNetV2-SSDLite, Tiny-DSOD, Pelee, YOLO-Lite, MnasNet-A1 + SSDLite, YOLOv3-Tiny, NanoDet and Mini YOLO are not efficient when deployed.

Additionally, in latest years, one-stage YOLO-based lightweight object detectors have been developed which are mentioned in Table 6. In 2024, DSP-YOLO (Zhang et al. 2024) and YOLO-NL (Zhou 2024) emerged but not ready to be deployed in real-life applications. On the contrary, EL-YOLO (Hu et al. 2023a, 2023b), YOLO-S (Betti and Tucci 2023) GCL-YOLO (Cao et al. 2023), Light YOLO (Yin et al. 2023), Edge YOLO (Li and Ye 2023), GL-YOLO-Lite (Dai and Liu 2023) and LC-YOLO (Cui et al. 2023) can be merged in real-life applications of computing world. Further, we have added performance parameters in terms of FLOPs, Inference time, FPS and number of parameters with respect to each latest YOLO-based light-weight object detector. YOLO-S utilized least number of FLOPs i.e., 34.59B whereas Light YOLO has maximum FPS of 102.04 and GCL-YOLO has lease number of parameters as depicted in Table 6.

3.3 Backbone architecture for deep learning-based lightweight object detection models

Deep learning-based models for image processing advanced and effectively outperformed more conventional methods in terms of object classification (Krizhevsky et al. 2012). The

Table 6 Performance parameters of latest YOLO-based lightweight object detectors

S. No.	Year	Light-weight detector	FLOPs	Inference time (ms)	FPS	Parameters (M)	Real-time applications
1	2024	DSP-YOLO	15.8G	–	71.4	7.02	*
2	2024	YOLO-NL	154.6G	10.4	96.1	54.76	*
3	2023	EL-YOLO	–	41.0	–	7.56	√
4	2023	YOLO-S	34.59B	–	75	7.8	√
5	2023	GCL-YOLO	10.7G	–	53	1.64	√
6	2023	Light YOLO	–	73	102.04	3.23	√
7	2023	Edge YOLO	14.2G	–	80.7	5.8	√
8	2023	GL-YOLO-Lite	3.3G	70.51	52.63	4.41	√
9	2023	LC-YOLO	17.6G	15.6	–	7.30	√

*—not currently, but aiming in future

most effective deep learning object categorization architectures have been Convolutional Neural Networks (CNNs), which function similarly to human brains and include neurons that react to their surroundings in real time (Makantasis et al. 2015; Fernández-Delgado et al. 2014). Well-known CNN architectures based on deep learning have been used for object classification-based feature extractors to fine-tune the classifiers. Forward propagation is used to process the training with random seeds for the filters and parameters. However, due to severely resource-constrained conditions, notably in memory bandwidth, the development of specialised CNN architectures for lightweight object identification models has received less attention than expected. In this section, we summarized backbones i.e., feature extractors for deep learning-based lightweight object detection models. Backbone architectures are used to extract the features for conducting lightweight object identification tasks where an image is provided as an input and a feature map is produced as an output. The majority of backbone architectures for detection tasks are essentially networks for classification problems, which take into account the final fully linked layers. DetNaS convolutional neural network is shown in Fig. 4 to help understand how backbone architectures function in the context of lightweight object identification models. These architectures are shown block-by-block. ShuffleNetv2 5*5 and 7*7 blocks are what the blue and green blocks are made of. Kernel sizes for the blue blocks are 3. In comparison to pink 3*3 ShuffleNetv2 blocks, the peach colour blocks are Xception ShuffleNetv2 blocks (Ma et al. 2018). Each level has eight blocks and the total number of blocks is forty. Large-kernel blocks are found in low-level layers while deep blocks are found in high-level layers in the lightweight DetNAS architecture. Blocks of huge kernels (5*5, 7*7) are present in DetNAS' stage 1 and stage 2's low-level layers. Pink-colored blocks, on the other hand, have kernels that are 3*3. Stages 3 and 4 are comprised of peach and pink blocks, as shown in the centre of Fig. 4. Six of these eight blocks—Xception and ShufflNetv2 blocks—are deeper than standard 3*3 ShufflNetv2 blocks. These results lead us to the conclusion that lightweight object detection networks differ visually from conventional detection and

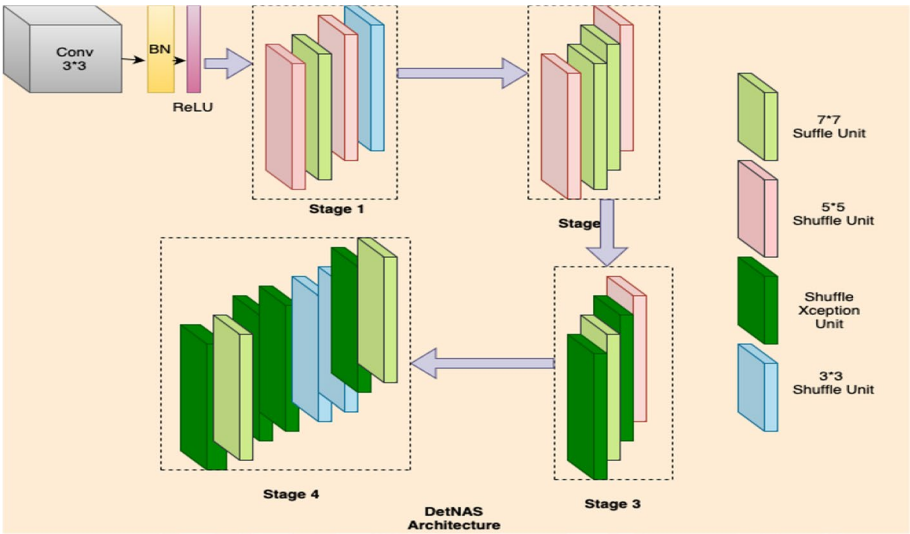

Fig. 4 Architectural details of backbone architecture DetNaS (Chen et al. 2019)

classification networks. In the next section, a brief introduction about deep learning-based lightweight backbone architectures have been given:

3.3.1 MobileNet (Howard et al. 2017)

MobileNet created an efficient network architecture made up of 28 depth-wise separable convolutions to factorise a standard convolution into a depth-wise convolution and a 1×1 point-wise convolution. By applying different kernels, isolating the filtering, and merging the features using pointwise convolution in depth-wise convolution, the computing cost and model size were reduced. Two more model-shrinking hyperparameters, width and resolution multiplier, were added in order to improve performance and reduce the size of the model. The model's oversimplification and linearity, which resulted in fewer channels for gradient flow, were corrected in later versions.

3.3.2 MobileNetV2 (Sandler et al. 2018)

The inverted residual with linear bottleneck, a novel module, was added to MobileNetv2 in order to speed up calculations and improve accuracy. In the MobileNetv2 there were two convolutional layers followed by 19 bottleneck modules. The computationally efficient MobileNetv2 feature extractor was used by the SSD writers to detect objects. With respect to the original SSD, the new device, known as SSDLite, touted an $8 \times$ reduction in parameters. It is simple to construct, generalises well to different datasets, and as a result, garnered positive feedback from the community.

3.3.3 MobileNetv3 (Qian et al. 2021)

In MobileNetv3, the unneeded portions of the network were iteratively removed during an automated platform search in a factorised hierarchical search space. This model is then modified to increase the desired metrics after the design proposal has been prepared. Since the architecture's filters regularly reflect one another, accuracy can be maintained even if half of them are discarded, which reduces the need for further processing. MobileNetv3 merged harsh Swish and RELU activation filters because the latter was computationally more efficient while preserving accuracy.

3.3.4 ShuffleNet (Zhang et al. 2018a, 2018b)

According to the authors, many effective networks lose their effectiveness as they scale down because of expensive 1×1 convolutions. ShuffleNet is a neural network design that was very computationally effective and created especially for mobile devices. To overcome the issue of restricted information flow, it was suggested to use group convolution along with channel shuffling. The ShuffleNet unit, like the ResNet block, substituted a pointwise group convolution for the 1×1 layer and a depth-wise convolution for the 3×3 layer.

3.3.5 ShuffleNetv2 (Ma et al. 2018)

ShuffleNetv2 advocated in favour of using speed or latency as direct measures rather than FLOPs or other indirect metrics to determine how complex a computation is. Four guiding principles served as its foundation: equal channel width to lower memory access costs, group convolution selection based on target platform, multi-path ways to boost accuracy, and element-wise operations. The input was split in half by a channel split layer in this model, and the residual link was concatenated by three convolutional layers before being sent through a channel shuffle layer. ShuffleNetv2 outperformed other cutting-edge models of comparable complexity, outperforming its peers.

3.3.6 PeleeNet (Wang et al. 2018)

PeleeNet is an inventive and effective architecture based on traditional convolution that was created using a number of computation-saving strategies. PeleeNet's design comprised four iterations of modified dense and transition layers, followed by the classification layer. The two-way dense layer helps to obtain distinct receptive field scales, which makes it simpler to identify larger things. Using a stem block minimised the loss of information. While our model's performance was not as good as modern object detectors on mobile and edge devices, it did demonstrate how even seemingly little design decisions can have a substantial impact on total performance.

3.3.7 mNASNet (Tan et al. 2019)

Using NAS (Neural Architecture Search) automation, mNASNet was created. It conceptualised the search problem as a multi-object optimisation problem, with a dual focus on latency and accuracy. Unlike previous models that stacked identical blocks, this allowed for the design of individual blocks. By dividing the CNN into distinct blocks and then looking for operations and connections in each of those blocks separately, it factorised the search space. mNASNet was roughly twice as rapid as MobileNetv2 and more accurate.

3.3.8 Once for all (OFA) (Cai et al. 2019)

In recent years, modern models have been constructed using NAS for architecture design; nonetheless, the sampled model training resulted in costly computations. This model just needs to be trained once, after which sub-networks can be constructed from it based on the requirements. Thanks to the OFA network, such sub-networks can be variable in the four key dimensions of a convolutional neural network: depth, width, kernel size, and dimension. They slowed down the training process and caused layering within the OFA network, which eventually resulted in gradual shrinkage.

3.3.9 MobileViT (Mehta and Rastegari 2021)

Combining the benefits of CNNs and Vision Transformers (ViT), MobileViT is a transformer-based detector that is lightweight, portable, and compatible with edge devices. It was able to successfully identify both short- and long-range dependencies by utilising

a unique MobileViT block. Alongside the MobileViT block, MobileNetv2 modules (Sandler et al. 2018) were made available in serial form. Unlike previous transformer-based networks, they used a transformer as a convolution, which automatically incorporated spatial bias, therefore location encoding was not necessary. MobileViT performed well on complex problems, supporting its claim to be a general-purpose backbone for various vision applications. Because of the constraints of transformers on mobile devices, it was able to attain better accuracy with a smaller parameter budget.

3.3.10 SqueezeNet (Iandola et al. 2016)

SqueezeNet attempts to maintain the accuracy of the network by using techniques with fewer parameters. Smaller filters, 3×3 filters for the input channels, and later network placement of the down-sampling layers were the design strategies employed. SqueezeNet's core module, the fire module, consisted of an extended layer and a squeeze layer, each containing a ReLU activation. Eight Fire modules were stacked and jammed in between the convolution layers to form the SqueezeNet architecture. Accuracy was increased over the basic model using SqueezeNet with residual connections, which was also developed and inspired by ResNet (He et al. 2016). SqueezeNet showed out as a serious contender for boosting the hardware efficiency of neural network topologies.

The year and initial usage in which backbone architectures have been utilized, the number of parameters, merits, and top-1 accuracy have been elaborated in the given Table 7. According to research on deep learning-based backbone architectures, SqueezeNet (Iandola et al. 2016) and ShuffleNetv2 (Ma et al. 2018) are the most widely used lightweight backbone architectures used in edge devices today. The performance of the model built from depth-wise separable convolutions, inverted residual topologies with linear bottlenecks, and automatic complementary search structures is gradually enhanced by the MobileNet series (Howard et al. 2017; Qian et al. 2021; Sandler et al. 2018).

4 Performance analysis of deep learning-based lightweight object detectors

In this section, a comprehensive analysis has been made from above-discussed lightweight object detectors and related backbone architectures. It can be said that lightweight object detectors based on deep learning strike a balance between accuracy and efficiency. Although the above-mentioned lightweight detectors from the previous sections have a quick inference rate, accuracy isn't always up to par for some jobs. As shown in Fig. 5, performance evaluation of deep learning-based lightweight object detectors in terms of mAP on MS-COCO dataset, lightweight object detector YOLOv7-x performs best among mentioned detectors. The backbone architectures in deep learning-based lightweight object detectors play a vital role in determining the accuracy of models. The convolutional architectures specifically designed for edge devices in terms of limited bandwidth usage would be an ideal choice for embedding in detection models. The top-1 accuracy comparison of deep learning-based lightweight backbone architectures in detection models is presented in Fig. 6. The backbone architecture ShuffleNetV2 attains 70.9 accuracy, a large jump from SqueezeNet (Iandola et al. 2016) results. A marginal accuracy increase can be seen in the architectures such as PeleeNet (Wang et al. 2018), DetNas (Chen et al. 2019), mNASNet

Table 7 Commonly utilized backbone architectures in deep learning-based lightweight object detection methods

S. No.	Year	Backbone Framework	Parameters	Inspired from	Top-1 accuracy	First used in	Merits
1	2017	SqueezeNet	4.8 M	ResNet	60.4	FPGA	50× fewer parameters than AlexNet
2	2018	ShuffleNetV2	5.2 M	ShuffleNet	70.9	Light-Head RCNN	Introduced a channel shuffle operation
3	2018	PeleeNet	5.9 M	DenseNet-41	72.1	SSD	Built with conventional convolution
4	2019	DetNas	5.2 M	ShuffleNetv2	73.8	RetinaNet FPN	Utilized NAS for the design of better backbones
5	2019	mNASNet	5.3 M	MobileNetV2	74.0	SSDLite	Incorporated model latency and factorized hierarchical search space, encourages layer diversity
6	2019	MobileNetV3	5.4 M	Platform-aware NAS	75.2	SSDLite	Allows fine-tuning of individual layers in a sequential manner
7	2020	OFA	7.7 M	NAS methods	76.0	FPGA	Reduce the cost by supporting diverse architectural settings
8	2020	GhostNet	5.2 M	ResNet	73.9	RetinaNet Faster RCNN	Apply cheap cost transformations to generate ghost feature maps
9	2022	MobileViT	6 M	Standard ViTs	78.4	SSD	Lightweight vision transformer for mobile devices

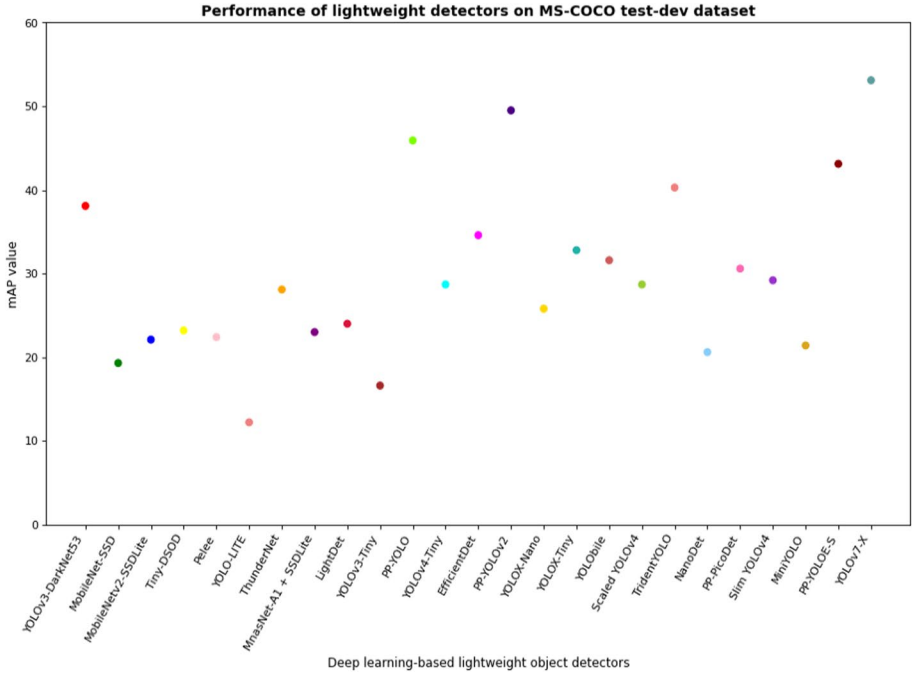

Fig. 5 mAP Performance evaluation of major deep learning-based lightweight object detectors

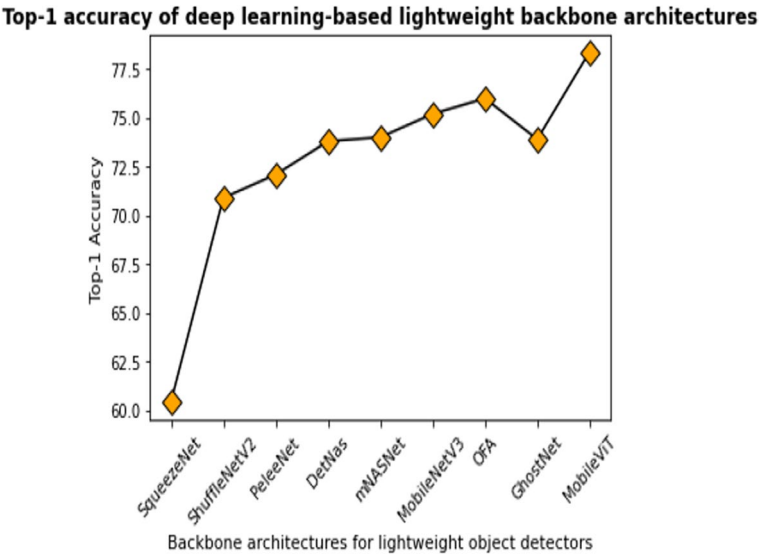

Fig. 6 Accuracy comparison of deep learning-based lightweight backbone architectures in detection models

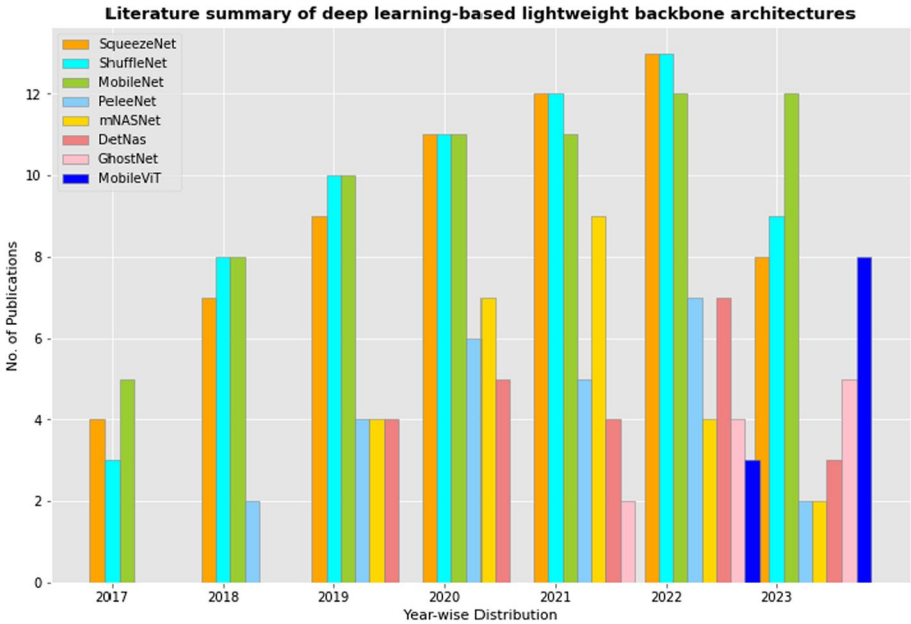

Fig. 7 Year-wise literature summary of backbone architectures in case of lightweight detection models

(Tan et al. 2019), GhostNet (Han et al. 2020a, 2020b) but the recently emerged transformers-based architecture, i.e., MobileViT (Mehta and Rastegari 2021) achieves best state-of-the-art results. Moreover, from year 2017 to 2023, we have shown literature summary in terms of number of publications for deep learning-based lightweight backbone architectures in Fig. 7. The most popular architecture SqueezeNet has been utilized over years in lightweight detectors as shown in Fig. 7. GhostNet (Paoletti et al. 2021) and MobileViT (Mehta and Rastegari 2021) backbone architectures have more literature in 2022 and 2023 year. As mentioned above, the state-of-the-art object detection works are either accuracy-oriented using a large model size (Ren et al. 2015; Liu et al. 2016; Bochkovskiy et al. 2020) or speed- oriented using a lightweight model but sacrificing accuracy (Wang et al. 2018; Sandler et al. 2018; Li et al. 2018; Liu and Huang 2018). It is difficult for any of existing lightweight detectors to meet the accuracy and latency requirements of real-world applications on mobile and edge devices at the same time. Therefore, we require a mobile device solution that can accomplish both high accuracy and low latency to deploy lightweight object detection models.

4.1 Benchmark detection databases for light-weight object detection models

In this section, most popular datasets have been discussed concerning to deep learning-based lightweight object detectors. Datasets are essential for lightweight object detection because they allow for standard comparisons of competing algorithms and the establishment of objectives for solutions.

4.1.1 PASCAL VOC (Everingham et al. 2010)

The most well-known object detection dataset is this one. The PASCAL-VOC versions VOC2007 and VOC2012 are frequently used in papers. 2501 training, 2510 validation, and 5011 testing images make up VOC2007. VOC2012, on the other hand, comprises of 10,991 training, 5823 validation images, and 5717 testing images. The PASCAL VOC datasets include 11,000 images spread across 20 visual object classes. Animals, vehicles, people, and domestic things are the four broad categories into which these 20 classes can be divided. Additionally, classifications of objects with semantic similarities, such as trucks and buses, enhance the complexity levels for detection. Visit http://host.robots.ox.ac.uk/pascal/VOC/to get the dataset.

4.1.2 MS-COCO (Lin et al. 2014)

A sizable image dataset called MS-COCO (Microsoft Common Objects in Context) contains 328,000 photographs of commonplace items and people. It is now one of the most well-liked and difficult object detection datasets. It has 897,000 tagged objects in 164,000 photos across 80 categories. For the training, validation, and testing sets, there are 118,287, 5000, and 40,670 photos, respectively. The distribution of objects in MS-COCO is more in line with real-world circumstances. There is no information available regarding the MS-COCO testing set's annotations. The following categories of annotations are offered by MS-COCO, including those for captioning, keypoints, panoptic segmentation, dense pose, and object detection. The MS-COCO dataset provides a wide range of realistic images, showing disorganized scenes with various backgrounds, overlapping objects, etc. The URL of the dataset is http://cocodataset.org.

4.1.3 KITTI (Geiger et al. 2013)

It is a well-known dataset for traffic scene analysis and includes 7518 photos for testing and 7481 for training that have been labelled. There are 100,000 pedestrian cases, 6000 IDs, and an average of one person per photograph. The pedestrian and cyclist are the two subclasses of the human class in KITTI. Based on how much the objects are obscured and shortened, the object labels are divided into easy, moderate, and hard levels. In this dataset, there are two subcategories for people: pedestrians and cyclists. Utilizing three criteria that differ in the minimum bounding box height and maximum occlusion level, the models trained on it are assessed. Visit http://www.cvlibs.net/datasets/kitti/index.php to download the dataset.

We have presented performance of deep learning-based lightweight detection models on above-discussed detection datasets in Fig. 8. The lightweight object detector YOLOv4-dense achieves mAP value of 84.30 on KITTI dataset, 71.60 on PASCAL VOC dataset. L4Net detector attains mAP value of 71.68 on KITTI, 82.30 on PASCAL VOC and 42.90 on MSCOCO dataset. RefineDet-lite detector achieves mAP value of 26.80 on MSCOCO dataset. Further to compare performances, FE_YOLO performs best on KITTI dataset as presented in Fig. 8 whereas L4Net detector performs best on MSCOCO dataset and finally, lightweight YOLO-Compact detector outperforms other detectors on PASCAL VOC dataset.

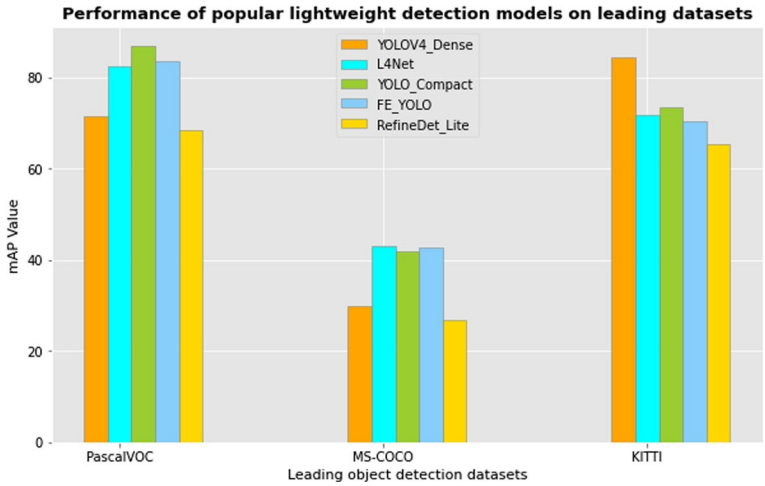

Fig. 8 Performance evaluation of deep learning-based lightweight models on leading datasets

4.2 Evaluation parameters

Lightweight object identification models based on deep learning use the same evaluation criteria as generic object detection models. Out of all predictions made, accuracy is the proportion of things that were successfully anticipated. When dealing with class unbalanced data, where the number of instances is not equal for each class, the accuracy result can be quite deceptive because it places more emphasis on learning the majority classes than the minority classes. Therefore, mean Average Precision (mAP), Frames Per Second, and the size of the model weight file serve as the primary evaluation indices for the effectiveness of the lightweight object identification model. The correct labelling data for each image provides the precise number of objects in each category in the image. Intersection Over Union (IoU) quantifies the similarity between the ground and predicted bounding box to evaluate how good the predicted bounding box is as represented in Eq. (1):

$$IoU_{pred}^{truth} = \frac{truth \cap pred}{truth \cup pred} \tag{1}$$

The calculation of IoU value takes place between each prediction box and ground data. Then consider the largest IoU value, and, based on the IoU threshold, we can calculate the number of True Positives (TP) and False Positives (FP) for each object category in an image. From this, the Precision of each category is calculated according to Eq. (2):

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

When the correct number of TP is obtained, the number of False Negatives (FN) are measured through Recall as in Eq. (3).

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

By figuring out various recall rates and associated accuracy rates for each category, PR curves for each can be plotted. The value of AP is identical to the region enclosed by the PR curve in the PASCAL VOC 2010 object detection competition evaluation criteria. Precision, recall rate, and average accuracy are three metrics that can be used to assess the model's accuracy for detecting tasks. MS COCO averages mAP with a step of 0.05 over a range of IoU thresholds (0.5, 0.55, 0.6, 0.65, 0.7, 0.75, 0.8, 0.85, 0.9, and 0.95). The main metric used to judge competitors is called "mAP," which averages AP over all 80 COCO dataset categories and all 10 criteria. A higher AP score according to the COCO evaluation criteria denotes flawless bounding box localization of the discovered items. The typical COCO-style AP metric, which averages APs over IoU threshold ranges of 0.5 to 0.95 with 0.05 steps. The performance is measured using AP50, AP75 at various IoU thresholds and AP_s, AP_m, and AP_l on objects that are small, medium, and large in size. By averaging over all 10 IoU thresholds across all categories with a uniform step size of 0:05, the primary metric, AP(IoU) = 0.50:0.05:0.95, is determined.

4.3 A summary of edge devices-based platforms for lightweight object detectors

In the upcoming years, a ton of data will be produced by mobile users and IoT devices. Data growth will bring new problems like latency. Additionally, traditional methods cannot be relied upon for very long if intelligence is to be derived from deep learning-based object detection and recognition algorithms in real-time. Edge computing devices have drawn a lot of interest as a result of prominent firms' efforts to make supercomputers affordable. It is vital to enable developers to swiftly design and deploy edge applications from lightweight detection models as the IoT, 5G, and portable processing device eras approach. As a result of advancements in the field of deep learning, numerous enhancements to object identification models have been presented that are aimed at edge device applications. DeepSense, TinyML, DeepThings, and, DeepIoT are just a few of the frameworks that have been published in recent years with the intention of compressing deep models for IoT edge devices. To satisfy the processing demands of deep learning-based lightweight object detectors, the model must be able to overcome several constraints like a limited battery, high energy consumption, limited computational capabilities, and a constrained memory while maintaining a level of accuracy. The primary goal should be to create a framework that makes it possible for machine learning models to be quickly implemented in Internet of Things devices. The well-known TinyML frameworks TensorFlow Lite from Google, ELL from Microsoft, ARM-NN and CMSIS-NN from ARM, STM 32Cube-Al from STMicroelectronics, and Alfes from Fraunhofer IMS enable the use of deep learning at the peripheral. When combined with other microcontroller-based tasks, low-latency, low-power, and low-bandwidth AI algorithms can function as part of an intelligent system at a low cost thanks to TinyML on a microcontroller. The DeepIoT framework reduces neural network designs into less dense matrices while preserving the performance of sensing applications by figuring out how few non-redundant hidden components, including filters and dimensions, are needed by each layer. Another well-known framework that provides deep learning-based lightweight object recognition is TensorFlow. TensorFlow Lite is a cross-platform, quick, and lightweight mobile and IoT framework (TFLite) to scale down their massive models. The majority of lightweight models employ TensorFlow lite quantization, which is easy to deploy on edge devices.

4.3.1 Mobile phones

The limitations imposed by mobile devices may be the reason why less research is being done on the deployment of object detectors on mobile phones than on other embedded platforms. Smartphone complexity and capabilities are rising quickly, but their size and weight are also probably going to decrease. Few literature studies have tried to perform implementation on smartphones-based devices (Lan et al. 2019; Liu et al. 2020a, 2020b, 2020c, 2020d; Liu et al. 2021a, 2021b, 2021c; Li et al. 2021a, 2021b, 2021c; Paluru et al. 2021). It can be seen that it puts a heavy burden on creating models that are small, light, and require a minimum number of computations. It is advised to test novel concepts for deep learning inference optimization on transportable models that are regularly utilized with cellphones (Xu et al. 2019). Either the spatial or temporal complexity of deep learning models can be reduced to the point where they can be fully implemented on mobile devices. But there may be a lot of security issues that need to be fixed Steimle et al. (2017). Although deep learning for smartphone item detection appears to be a promising field of study, success will need many more contributions (Wang et al. 2022a, 2022b, 2022c, 2022d).

4.3.2 IoT edge devices

Another way to enable deep learning on IoT edge devices is to transfer model inference to a cloud server. Another way to boost the power of these inexpensive devices is to add an accelerator. The price of using these accelerators is a major drawback, though. Some edge devices, like the Raspberry Pi, may require an extra accelerator, but some, like the Coral Dev Board, already have edge TPU accelerators built in. Deep learning can be more easily enabled to run locally or remotely using a distributed design that links computationally inefficient front-end devices with more potent back-end devices, like a cloud server or accelerator (Ran et al. 2017).

4.3.3 Embedded boards

To provide the finest design options, processor-FPGA combinations, and FPGAs with hard processor cores embedded into its fabric are widely used. Lattice Semiconductor, Xilinx, Microchip, and Altera from Intel are the well-known manufacturers. The literature suggests that the Xilinx boards family is the one that is most frequently utilized for deep learning-based applications. An additional accelerator is often needed when employing FPGA devices (Saidi et al. 2021) to get acceptable performance. Due to Integrated Development Environment (IDE) and high-level language support, the Arduino and Spark-based boards at the top of the device family allow for greater software level programming (Kondaveeti et al. 2021).

4.4 Applications specific to deep learning-based lightweight object detectors

In the above sections, we have discussed architectural details, leading datasets of deep learning-based lightweight object detection models. These models offer a multitude of applications such as in remote-sensing (Xu and Wu 2021; Ma et al. 2023), and aerial images (Xu and Wu 2021; Zhou et al. 2022), traffic monitoring (Jiang et al. 2023;

Zheng et al. 2023), fire detection (Chen et al. 2023), indoor robots (Jiang et al. 2022), pedestrian detection (Jian et al. 2023) etc. A summary of literature findings for supporting the applications of deep learning-based lightweight object detection models is listed in Table 6. In (Zhou et al. 2019), for Range Doppler (RD) radar pictures, a lightweight detection network called YOLO-RD has been proposed. Additionally, a brand-new, lightweight mini-RD dataset has been created for effective network training. On the mini-RD dataset, YOLO-RD produced effective results with a smaller memory budget and a detection accuracy of 97.54%. Regarding both algorithm and hardware resource aspects in object detection, (Ding et al. 2019) introduced REQ-YOLO, a resource conscious, systematic weight quantization framework for object detection. For non-convex optimisation problems on FPGAs, it applied the block-circulant matrix approach and proposed a heterogeneous weight quantization. The outcomes demonstrated that the REQ-YOLO framework can greatly reduce the size of the YOLO model while just slightly reducing accuracy. It is suggested that autonomous vehicles use the L4Net Locating object suggestions from (Wu et al. 2021) which integrates a key point detection backbone with a co-attention strategy by attaining cheaper computation costs with improved detection accuracy under a variety of resource constraints. To generate more precise prediction boxes, the backbone capture context-wise information and co-attention method specifically combined the strength of both class-agnostic and semantic attention. With a 13.7 M model size and speeds of 149 FPS on an NVIDIA TX and 30.7 FPS on a Qualcomm-based device, respectively, L4Net achieved 71.68% mAP. The development of effective object detectors requires the rapid development of CPU-only hardware because to the huge data processing and resource-constrained scenarios on GPUs. With three orthogonal training strategies—IoU-guided loss, classes-aware weighting method, and balanced multi-task training approach, (Chen et al. 2020a, 2020b) proposed a lightweight backbone and light-head detecting component. On a single-thread CPU, the suggested RefineDetLite obtained 26.8 mAP at a pace of 130 ms/pic. LiraNet, a compact CNN, was suggested by (Long et al. 2020a, 2020b) for the recognition of marine ship objects in radar pictures. By creating Lira-YOLO, a compact model that is simple to set up on mobile devices, LiraNet was mounted on the already-existing detection framework Darknet. Additionally, a lightweight dataset of distant Doppler domain radar pictures known as mini-RD had been created to test the performance of the proposed model. Studies reveal that Lira-YOLO's network complexity is minimal at 2.980 Bflops, and its parameter quantity is reduced at 4.3 MB thanks to its high detection accuracy of 83.21%. (Lu et al. 2020) developed a successful YOLO-compact network for real-time object detection in the single person category. The down sampling layer was separated in this network, which facilitated the modular design by enhancing the remaining bottleneck block. YOLO-compact's AP result is 86.85% and its model size is 9 MB, making it smaller than tiny-yolov3, tiny-yolov2, and YOLOv3. By focusing on small targets and background complexity, (Xu and Wu 2021) presented FE-YOLO for deep learning-based target detection from remote sensing photos. The analyses on remote sensing datasets demonstrate that our suggested FE-YOLO outperformed existing cutting-edge target detection methods. A brand-new YOLOv4-dense model was put forth by (Jiang et al. 2023) for real-time object recognition on edge devices. To address the issue of losing small objects and further minimize the computing complexity, a dense block had been devised. With 20.3 M parameters, YOLOv4-dense obtained 84.3% mAP and 22.6 FPS. To improve the detection of small and medium-sized objects in aerial photos, (Zhou et al. 2022) developed Dense Feature Fusion Path Aggregation Network (DFF-PANet). The trials were conducted using the HRSC2016 dataset and

the DOTA dataset, yielding 71.5% mAP and 9.2 M as a lightweight model. To help an indoor mobile robot solve the problem of object detection and recognition, (Jiang et al. 2022) presented ShuffleNet-SSD. Deep separable convolution, point-by-point grouping convolution, and channel rearrangement were all created using the suggested model. A dataset has been created for the mobility robot under the indoor scene. For the detection of dead trees, (Wang et al. 2022a, 2022b, 2022c, 2022d) suggested a novel, lightweight architecture called LDS-YOLO based on the YOLO framework. These plants assisted in the timely regeneration of dead trees, allowing the ecosystem to remain stable and efficiently withstand catastrophic disasters. With the addition of the SoftPool approach in Spatial Pyramid Pooling (SPP), a unique feature extraction module is provided that makes use of the features from earlier layers in order to ensure that small targets are not ignored. On the basis of UAV-captured photos, the suggested approach is assessed, and the experimental findings show that the LDS-YOLO architecture works well when compared to AP of 89.11% and parameter size of 7.6 MB. The categorization of several applications concerning to lightweight object detectors as shown in Table 8 with respect to image type such as remote-sensing, aerial, medical and video streams and application type of healthcare, medical, military and industrial use.

4.5 Discussion and contributions

According to the above-mentioned analysis of deep learning-based light-weight object detectors, there is a need of focus to develop detectors for edge devices which can strike a good balance between speed and accuracy. Furthermore, real-time deployment of these detectors on edge devices is also needed while achieving accuracy of lightweight detectors without compromising precision. In 2022, lightweight backbone architectures ShuffleNet and SqueezeNet have highest publications with respect to lightweight object detectors. In 2023, transformers based MobileViT started getting attention of researchers as top-1 accuracy of 78.4 is also achieved and MobileNet backbone architectures were maximum employed when compared with others. As shown in Table 8, according to input type, video streams have maximum employability in deep learning-based light-weight object detectors. With respect to diverse applications, traffic and pedestrians related detection problems, obstacles and driving assistance have highest studies whereas all other existing applications have limited light weight detectors on edge devices. As we witnessed, the majority of presented light-weight models are from the YOLO family, a number of deep network layers with increasing number of parameters needing to account for the improved accuracy. Therefore, the most important question to ask when a model migrates from a cloud device to an edge device is how to lower the parameters of a deep learning-based lightweight model. There are numerous approaches being used to address this which are described in the next section.

4.6 Recommendations for designing powerful deep learning-based lightweight models

Researchers have created new training methods that decrease the memory footprint in the edge device and speed up training on low-resource devices, in addition to specialized hardware for the deep learning model training process at the network edge. The techniques which we discussed in this section- pruning, quantification, knowledge distillation, and low-rank decomposition, are the four key categories used to compress pre-training

Table 8 A list of type-based applications with respect to deep learning lightweight object detection models

S. No.	Type	Lightweight object model applications	Studies
1	According to input images	Remote-sensing/hyperspectral images	Xu and Wu (2021), Huyan et al. (2021), Shi et al. (2020), Liang and Wang (2021), Xu et al. (2021), Pang et al. (2019), Yin et al. (2020)
		UAV/aerial images	Zhou et al. (2022), Tang et al. (2020a, 2020b), Jin and Lin (2019), Li et al. (2021a, 2021b, 2021c), Kyrkou and Theocharides (2020), Guo et al. (2020)
		CT/ Ultrasound /MRI/ X-rays/	Paluru et al. (2021), Gadosey et al. (2020), Li et al. (2021a, 2021b, 2021c), Zhou et al. (2021a, 2021b), Zhang et al. (2020), Kong et al. (2020a, 2020b)
		Video Streams	Panero et al. (2021), Jeong et al. (2020), Cho et al. (2020), Zhang et al. (2019a, 2019b), Liu et al. (2020a, 2020b, 2020c, 2020d), Mansouri et al. (2020), Silva et al. (2019), Rani (2021), Rui et al. (2019), Tzelepi and Tefas (2020), Alfasly et al. (2019), Lai et al. (2021), Muhammad et al. (2019), Wu et al. (2019), Bai and Zhou (2020), Hou et al. (2021), Liu et al. (2021a, 2021b, 2021c), Zhang et al. (2020), Chang et al. (2021), Cheng et al. (2020), Kyrkou (2021), Ke et al. (2021)

Table 8 (continued)

S. No.	Type	Lightweight object model applications	Studies
2	According to diverse applications	Transportation in lane and licence plate detection, cars, sleepiness detection, pedestrian and obstacle detection, and driving assistance	Jiang et al. (2023), Zheng et al. (2023), Wu et al. (2021), Qin and Liu (2020), Kyrkou (2020), Luo et al. (2019), Haque et al. (2021), He et al. (2020), Zhao et al. (2020a, 2020b, 2020c), Kim et al. (2019), Ma et al. (2019), Ullah et al. (2021), Tsai et al. (2021), Hu et al. (2021), Ke et al. (2021), Chen et al. (2020a, 2020b), Ouyang et al. (2019), Wang et al. (2020), Han et al. (2019), Ping et al. (2023), Liu et al. (2021a, 2021b, 2021c)
		Plant disease detection, food production, crop harvesting, pest management, intrusion detection, and intelligent farming	Paoletti et al. (2021), Li et al. (2021a, 2021b, 2021c), Yin et al. (2020), Guo et al. (2020), Cui et al. (2021), Wang et al. (2021), Hu et al. (2021), Kamal et al. (2019), Kang et al. (2020), Wang et al. (2022a, 2022b, 2022c, 2022d)
		Target detection, flight aid, military instruction, and danger detection	Ma et al. (2019), Yuan et al. (2019), Gagliardi et al. (2021), Yuan et al. (2019)
		Detection and diagnosis of malignancies or tumors, assistance with surgery and instruction, examination of cellular body structures, and supervision and care of patient health	Paluru et al. (2021), Gadosey et al. (2020), Li et al. (2021a, 2021b, 2021c), Zhou et al. (2021a, 2021b), Zhang et al. (2020), Kong et al. (2020a, 2020b), Samore et al. (2020), Sun et al. (2021), Jiang et al. (2021), Long (2020), Shoeibi et al. (2023), Zhu et al. (2023)
		Giving access to virtual assistants, fault detection, intelligent factories, and industrial robot vision	Long et al. (2020a, 2020b), Zhang et al. (2019a, 2019b), Zhao et al. (2020c), Muhammad et al. (2019), Li et al. (2020a, 2020b), Dai et al. (2021), Mansouri et al. (2020), Liu et al. (2020a, 2020b, 2020c, 2020d, 2022a, 2022b), Han et al. (2020a, 2020b), Ophoff et al. (2019), Song et al. (2021)
3	Others	Object detection on FPGAs, CPU only devices, Indoor mobile robots	Ding et al. (2019), Jiang et al. (2022), Chen et al. (2020a, 2020b), Lu et al. (2020)

networks (Kamath and Renuka 2023) and listed in the following (Koubaa et al. 2021; Makkar et al. 2021; Wang et al. 2020a, 2020b, 2020c):

4.6.1 Pruning

Network pruning is a useful technique for reducing the size of the object detection model and speeding up model reasoning. By cutting out connections between neurons that are irrelevant to the application, this method lowers the amount of computations needed to analyse fresh input. In addition to eliminating connections, it can also eliminate neurons that are deemed irrelevant when the majority of their weights are low in relation to the deep neural network's overall context. With the use of this method, a deep neural network with reduced size, greater speed, and improved memory efficiency can be used in low-resource devices, such as edge devices.

4.6.2 Weights quantization

The weight quantization approach, which trades precision for speed, shrinks the model's storage capacity by reducing the number of floating-point parameters. In addition to eliminating pointless associations, every weight is kept as separate values. The weights quantization technique aims to compress these values to integers or numbers that occupy as few bits as possible by clustering related weight values into a single value. Consequently, there will be a re-adjustment of the weights, indicating a modification of the precision as well. This results in a cyclical implementation where the weights are quantified following each training.

4.6.3 Knowledge distillation

Dissection of knowledge presents itself as a new mode of transfer learning. This technique can extract knowledge from a big and well-trained deep neural network, dubbed teacher in this case, into a reduced deep network, called student. By doing this, the student network can learn to achieve the same outcomes as the teacher network while decreasing in size and increasing processing speed. Through the process of knowledge distillation, information is transferred from a large, thoroughly trained end-to-end detection network to numerous, quicker sub-models.

4.6.4 Training tiny networks

The deep neural network's initial convolution kernel is mostly broken-down using matrix decomposition in the low-rank decomposition method, although the accuracy of the results will noticeably improve. Directly training tiny networks can drastically reduce network accuracy loss and speed up reasoning.

4.6.5 Federated learning and model partition

Distributed learning or federated learning are two possible training approaches for dealing with complicated tasks or a period of training including a lot of data. The data would be broken into smaller groups that would be distributed among the nodes of the edge network. As part of the final deep neural network, each node would train based on the

data it received, enabling active learning capabilities at the network edge. Model partitioning is a strategy that may be applied in the inferring phase using the same methodology. To divide the burden, a separate node would compute each layer of the deep neural network in a model split. This approach would also make scaling simple.

Moreover, to boost the flow of information in a constrained amount of time, multi-scale feature learning in lightweight detection models that comprise single feature maps, pyramidical feature hierarchies, and integrated features may be used. The feature pyramid networks, as well as their variations such as feature fusion, feature pyramid generation, and multi-scaled fusion module, aid in overcoming object detection difficulties. Additionally, in order to boost the effectiveness of lightweight object identification models, researchers also work to encourage the development of activation functions and normalization in various applications. Above-mentioned techniques accelerate the usage of deep learning models into edge devices. The deep learning-based lightweight object detection models have not yet achieved comparable results when compared with generic object detection. Moreover, to mitigate these differences, a need for designing powerful and innovative lightweight detectors is a must. Some recommendations for designing powerful lightweight deep learning-based detectors are mentioned in this section.

1. **Incorporation of FPNs**- The bidirectional FPN can be utilized to improve the semantic information while incorporating feature fusion operations (Wang et al. 2023a, 2023b). To successfully collect bottom-up and top-down features more than FPN, an effective feature-preserving and refining module can be introduced (Tang et al. 2020a, 2020b). Deep learning-based lightweight detectors can be designed with the help of cross-layer connections and the extraction of features at various sizes while using depth-wise separable convolution. It is possible to take advantage of a multi-scale FPN architecture with a lightweight backbone to take out features from the input image.
2. **Transformer-based Solutions**- To increase the precision of the transformer-based lightweight detectors, group normalisation can be implemented in the encoder-and-decoder module and h-sigmoid activation function in the multi-layer perceptron (Li, Wang and Zhang 2022).
3. **Receptive Fields Enlargement**- The capacity of single-scale features to express themselves and to be detected on a single scale are both improved by the multi-branch block involving various receptive fields. The network width may increase and performance may be slightly enhanced with the use of several network branches (Liu et al. 2022).
4. **Feature Fusion Operation**- In order to combine several feature maps of the backbone and the collection of multi-scale features into a feature pyramid, the fusion operation offers a concatenation model (Mao et al. 2019). To improve the extraction of information from the suggested lightweight model, the feature maps' weight of various channels can be reassigned. Furthermore, performance improvement may result from the integration of the attention module and data augmentation technique (Li et al. 2022a, 2022b). The smooth fusion of semantic data from low-resolution scale to neighbourhood high-resolution scale is made possible by the implementation of FPN into the suggested lightweight detector architecture (Li et al. 2018).
5. **Effect of Depth-wise Separable Convolution**- The optimal design principle for lightweight object detection models consists of fewer channels with more convolutional layers (Kim et al. 2016). The approach to network scaling that modifies width, resolution, and network's structure to reduce or balance the size of the feature map, keep the number of channels constant after convolution, and minimise convolutional input and output is

where researchers can concentrate (Wang et al. 2021b). The typical convolution in the network structure can be replaced with an over-parameterized depth-wise convolutional layer, which significantly reduces computation and boosts network performance. To increase the numerical resolution, ReLU6 can be used in place of the activation function known as Leaky ReLU (Ding et al. 2022).

6. **Increase in Semantic Information**- To keep semantic features and high-level feature maps in the deep lightweight object network, the proposal of smaller cross-stage partial SPPs and RFBs facilitates the integration of high-level semantic information with low-level feature maps (Wang et al. 2022a, 2022b, 2022c, 2022d). The architectural additions of the context enhancement and spatial attention module can be employed to generate more discriminative feature representation (Qin et al. 2019).
7. **Pruning Strategy**- Block-punched pruning uses a fine-grained structured pruning method to maximise structural flexibility and minimise accuracy loss. High hardware parallelism can be achieved using the block-punched pruning strategy if the block size is suitable and compiler-level code generation is used (Cai et al. 2021).
8. **Assignment Strategy-** To improve the training of lightweight object detectors based on deep learning, use the SIMOTA dynamic label assignment method. When creating lightweight detection models, the combination of the regression method based on FCOS, dynamic and learnable sample assignment, and varifocal loss handling class imbalance works better (Yu et al. 2021). Designing lightweight object detectors using the anchor-free approach has been successful when combined with other cutting-edge detection methods using decoupled heads and the top label assignment strategy, known as SimOTA (Ge et al. 2021).

There are two ways of deployment of deep learning-based lightweight models on edge devices are when a light-weight model or compressed data are employed to match the compute capabilities of the limited edge devices. With regard to on-board object detection, this is true. The compromise between compression ratio and detection accuracy in this method is its drawback. Secondly, the model is distributed and data is exchanged when computations are spread over several devices and cloud server could be able to handle the computations in this situation. In this case, privacy and security seem to be the primary issues (Zhang et al. 2020a, 2020b, 2020c). Consideration must be given when establishing device coordination in this scenario as it may also result in extra overhead in order to avoid the edge devices being overworked while conducting the collaborative learning algorithm. No matter the plan, all of these deployment methods rely on edge devices and have to deal with the problems with edge devices present. The primary causes of the issue are data disparity in real-world scenarios and the need to manage real-time sensor data while performing numerous deep learning tasks. The powerful processing units, the high computing requirements of deep learning models, and short battery life makes validity of light-weight models tough. In the future, we'll strive to create such standards-compliant light-weight detection deployment models.

5 Conclusion

This study asserted that deep learning-based lightweight object detection models are a good candidate for improving the hardware efficiency of neural network architectures. This survey has examined and provided the most recent lightweight edge gadget models.

The commonly utilized backbone architectures in deep learning-based lightweight object detection methods have also been stated in which ShuffleNet and MobileNetV2 employed majorly in these models. Some critical aspects after analyzing current state-of-the-art deep learning-based lightweight object detection models on edge devices have been discussed. The comparison has been drawn between emerging lightweight object detection models on the basis of COCO-based mAP scores. The presentation of a summary of heterogeneous applications for lightweight object identification models that take into account diverse types of photos and application categories. This study also gives information on edge platforms for using portable detector models. A few recommendations are also given for creating a potent deep learning-based lightweight model, including multi-scale and multi-branch FPNs, federated learning, partitioning strategy, pruning, knowledge distillation, and label assignment algorithms. The lightweight detectors still fall more than 50% short in delivering such outcomes, although having demonstrated significant potential by matching classification errors with the thorough models.

Author contributions I, Payal Mittal is the sole author of this manuscript.

Declarations

Competing Interests The authors declare no competing interests.

Consent for publication During the preparation of this work the author has not used Generative AI and AI-assisted technologies in writing of this manuscript. The author solely reviewed and edited the content manually as needed and takes full responsibility for the content of the publication.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Abou El Houda Z, Brik B, Ksentini A, Khoukhi L (2023) A MEC-based architecture to secure IOT applications using federated deep learning. IEEE Internet Things Mag 6(1):60–63

Agarwal S, Terrail JOD, Jurie F (2018) Recent advances in object detection in the age of deep convolutional neural networks. arXiv preprint arXiv:1809.03193

Alfasly S, Liu B, Hu Y, Wang Y, Li CT (2019) Auto-zooming CNN-based framework for real-time pedestrian detection in outdoor surveillance videos. IEEE Access 7:105816–105826

Bai X, Zhou J (2020) Efficient semantic segmentation using multi-path decoder. Appl Sci 10(18):6386

Betti A, Tucci M (2023) YOLO-S: a lightweight and accurate YOLO-like network for small target detection in aerial imagery. Sensors 23(4):1865

Bochkovskiy A, Wang CY, Liao HYM (2020) Yolov4: optimal speed and accuracy of object detection. arXiv preprint arXiv:2004.10934

Brunetti A, Buongiorno D, Trotta GF, Bevilacqua V (2018) Computer vision and deep learning techniques for pedestrian detection and tracking: a survey. Neurocomputing 300:17–33

Cai Z, Vasconcelos N (2018) Cascade r-cnn: delving into high quality object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 6154–6162)
Cai H, Gan C, Wang T, Zhang Z, Han S (2019) Once-for-all: Train one network and specialize it for efficient deployment. arXiv preprint arXiv:1908.09791
Cai Y, Li H, Yuan G, Niu W, Li Y, Tang X, Ren B, Wang Y (2021) Yolobile: real-time object detection on mobile devices via compression-compilation co-design. In Proceedings of the AAAI conference on artificial intelligence (Vol. 35, No. 2, pp. 955–963)
Cao J, Bao W, Shang H, Yuan M, Cheng Q (2023) GCL-YOLO: a GhostConv-based lightweight yolo network for UAV small object detection. Remote Sens 15(20):4932
Chabas JM, Chandra G, Sanchi G, Mitra M (2018) New demand, new markets: What edge computing means for hardware companies. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/new-demand-new-markets-what-edge-computing-means-for-hardware-companies
Chang L, Zhang S, Du H, You Z, Wang S (2021) Position-aware lightweight object detectors with depthwise separable convolutions. J Real-Time Image Proc 18:857–871
Chen Y, Yang T, Zhang X, Meng G, Xiao X, Sun J (2019) Detnas: backbone search for object detection. Adv Neural Inf Process Syst. https://doi.org/10.48550/arXiv.1903.10979
Chen L, Ding Q, Zou Q, Chen Z, Li L (2020b) DenseLightNet: a light-weight vehicle detection network for autonomous driving. IEEE Trans Industr Electron 67(12):10600–10609
Chen C, Yu J, Lin Y, Lai F, Zheng G, Lin Y (2023) Fire detection based on improved PP-YOLO. SIViP 17(4):1061–1067
Chen C, Liu M, Meng X, Xiao W, Ju Q (2020) Refinedetlite: a lightweight one-stage object detection framework for cpu-only devices. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops (pp. 700–701)
Cheng Y, Li G, Wong N, Chen HB, Yu H (2020) DEEPEYE: a deeply tensor-compressed neural network for video comprehension on terminal devices. ACM Trans Embed Comput Syst (TECS) 19(3):1–25
Cho C, Choi W, Kim T (2020) Leveraging uncertainties in Softmax decision-making models for low-power IoT devices. Sensors 20(16):4603
Cui B, Dong XM, Zhan Q, Peng J, Sun W (2021) LiteDepthwiseNet: a lightweight network for hyperspectral image classification. IEEE Trans Geosci Remote Sens 60:1–15
Cui M, Gong G, Chen G, Wang H, Jin M, Mao W, Lu H (2023) LC-YOLO: a lightweight model with efficient utilization of limited detail features for small object detection. Appl Sci 13(5):3174
Dai Y, Liu W (2023) GL-YOLO-Lite: a novel lightweight fallen person detection model. Entropy 25(4):587
Dai W, Li D, Tang D, Jiang Q, Wang D, Wang H, Peng Y (2021) Deep learning assisted vision inspection of resistance spot welds. J Manuf Process 62:262–274
Dai J, Li Y, He K, Sun J (2016) R-fcn: Object detection via region-based fully convolutional networks. Advances in neural information processing systems, 29
Detector AFO (2022) Fcos: a simple and strong anchor-free object detector. IEEE Transactions on Pattern Analysis and Machine Intelligence, 44(4)
Dey S, Mukherjee A (2018) Implementing deep learning and inferencing on fog and edge computing systems. In 2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops) (pp. 818–823). IEEE
Ding P, Qian H, Chu S (2022) Slimyolov4: lightweight object detector based on yolov4. J Real-Time Image Proc 19(3):487–498
Ding C, Wang S, Liu N, Xu K, Wang Y, Liang Y (2019) REQ-YOLO: a resource-aware, efficient quantization framework for object detection on FPGAs. In proceedings of the 2019 ACM/SIGDA international symposium on field-programmable gate arrays (pp. 33–42)
Drolia U, Guo K, Narasimhan P (2017) Precog: prefetching for image recognition applications at the edge. In Proceedings of the Second ACM/IEEE Symposium on Edge Computing (pp. 1–13)
Duan K, Bai S, Xie L, Qi H, Huang Q, Tian Q (2019) Centernet: keypoint triplets for object detection. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 6569–6578)
Everingham M, Van Gool L, Williams CK, Winn J, Zisserman A (2010) The pascal visual object classes (VOC) challenge. Int J Comput vis 88:303–338
Fernández-Delgado M, Cernadas E, Barro S, Amorim D (2014) Do we need hundreds of classifiers to solve real world classification problems? J Mach Learn Res 15(1):3133–3181
Gadosey PK, Li Y, Agyekum EA, Zhang T, Liu Z, Yamak PT, Essaf F (2020) SD-UNET: stripping down U-net for segmentation of biomedical images on platforms with low computational budgets. Diagnostics 10(2):110
Gagliardi A, de Gioia F, Saponara S (2021) A real-time video smoke detection algorithm based on Kalman filter and CNN. J Real-Time Image Proc 18(6):2085–2095

Ge Z, Liu S, Wang F, Li Z, Sun J (2021) Yolox: exceeding yolo series in 2021. arXiv preprint arXiv:2107.08430

Geiger A, Lenz P, Stiller C, Urtasun R (2013) Vision meets robotics: the kitti dataset. Int J Robot Res 32(11):1231–1237

Girshick R (2015) Fast r-cnn. In Proceedings of the IEEE international conference on computer vision (pp. 1440–1448)

Guo W, Li W, Li Z, Gong W, Cui J, Wang X (2020) A slimmer network with polymorphic and group attention modules for more efficient object detection in aerial images. Remote Sens 12(22):3750

Han J, Zhang D, Cheng G, Liu N, Xu D (2018) Advanced deep-learning techniques for salient and category-specific object detection: a survey. IEEE Signal Process Mag 35(1):84–100

Han S, Yoo J, Kwon S (2019) Real-time vehicle-detection method in bird-view unmanned-aerial-vehicle imagery. Sensors 19(18):3958

Han S, Liu X, Han X, Wang G, Wu S (2020b) Visual sorting of express parcels based on multi-task deep learning. Sensors 20(23):6785

Han K, Wang Y, Tian Q, Guo J, Xu C, Xu C (2020) Ghostnet: More features from cheap operations. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 1580–1589)

Haque WA, Arefin S, Shihavuddin ASM, Hasan MA (2021) DeepThin: a novel lightweight CNN architecture for traffic sign recognition without GPU requirements. Expert Syst Appl 168:114481

He W, Huang Y, Fu Z, Lin Y (2020) Iconet: a lightweight network with greater environmental adaptivity. Symmetry 12(12):2119

He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 770–778)

He K, Gkioxari G, Dollár P, and Girshick R (2017) Mask r-cnn. In Proceedings of the IEEE international conference on computer vision (pp. 2961–2969)

Hou Y, Li Q, Han Q, Peng B, Wang L, Gu X, Wang D (2021) MobileCrack: object classification in asphalt pavements using an adaptive lightweight deep learning. J Trans Eng Part b: Pavements 147(1):04020092

Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, Andreetto M, Adam H (2017) Mobilenets: efficient convolutional neural networks for mobile vision applications. arXiv preprint arXiv:1704.04861

Hu X, Yang W, Wen H, Liu Y, Peng Y (2021) A lightweight 1-D convolution augmented transformer with metric learning for hyperspectral image classification. Sensors 21(5):1751

Hu M, Li Z, Yu J, Wan X, Tan H, Lin Z (2023b) Efficient-lightweight yolo: improving small object detection in yolo for aerial images. Sensors 23(14):6423

Hu B, Wang Y, Cheng J, Zhao T, Xie Y, Guo X, Chen Y (2023) Secure and efficient mobile DNN using trusted execution environments. In Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security (pp. 274–285)

Hua H, Li Y, Wang T, Dong N, Li W, Cao J (2023) Edge computing with artificial intelligence: a machine learning perspective. ACM Comput Surv 55(9):1–35

Huang Z, Yang S, Zhou M, Gong Z, Abusorrah A, Lin C, Huang Z (2022) Making accurate object detection at the edge: review and new approach. Artif Intell Rev 55(3):2245–2274

Huang L, Yang Y, Deng Y, Yu Y (2015) Densebox: unifying landmark localization with end to end object detection. arXiv preprint arXiv:1509.04874

Huang R, Pedoeem J, Chen C (2018) YOLO-LITE: a real-time object detection algorithm optimized for non-GPU computers. In 2018 IEEE international conference on big data (big data) (pp. 2503–2510). IEEE

Huang X, Wang X, Lv W, Bai X, Long X, Deng K, Dang Q, Han S, Liu Q, Hu X, Yu D (2021) PP-YOLOv2: a practical object detector. arXiv preprint arXiv:2104.10419

Huyan L, Bai Y, Li Y, Jiang D, Zhang Y, Zhou Q, Wei J, Liu J, Zhang Y, Cui T (2021) A lightweight object detection framework for remote sensing images. Remote Sens 13(4):683

Iandola FN, Han S, Moskewicz MW, Ashraf K, Dally WJ, Keutzer K (2016) SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and< 0.5 MB model size. arXiv preprint arXiv:1602.07360

Isereau D, Capraro C, Cote E, Barnell M, Raymond C (2017) Utilizing high-performance embedded computing, agile condor, for intelligent processing: An artificial intelligence platform for remotely piloted aircraft. In 2017 Intelligent Systems Conference (IntelliSys) (pp. 1155–1159). IEEE

Jain DK, Zhao X, González-Almagro G, Gan C, Kotecha K (2023) Multimodal pedestrian detection using metaheuristics with deep convolutional neural network in crowded scenes. Inf Fus 95:401–414

Jeong M, Park M, Nam J, Ko BC (2020) Light-weight student LSTM for real-time wildfire smoke detection. Sensors 20(19):5508

Jiang S, Li H, Jin Z (2021) A visually interpretable deep learning framework for histopathological image-based skin cancer diagnosis. IEEE J Biomed Health Inform 25(5):1483–1494
Jiang L, Nie W, Zhu J, Gao X, Lei B (2022) Lightweight object detection network model suitable for indoor mobile robots. J Mech Sci Technol 36(2):907–920
Jiang Y, Li W, Zhang J, Li F, Wu Z (2023) YOLOv4-dense: a smaller and faster YOLOv4 for real-time edge-device based object detection in traffic scene. IET Image Proc 17(2):570–580
Jiang Z, Zhao L, Li S, Jia Y (2020) Real-time object detection method based on improved YOLOv4-tiny. arXiv preprint arXiv:2011.04244
Jiao L, Zhang F, Liu F, Yang S, Li L, Feng Z, Qu R (2019) A survey of deep learning-based object detection. IEEE Access 7:128837–128868
Jin R, Lin D (2019) Adaptive anchor for fast object detection in aerial image. IEEE Geosci Remote Sens Lett 17(5):839–843
Jin Y, Cai J, Xu J, Huan Y, Yan Y, Huang B, Guo Y, Zheng L, Zou Z (2021) Self-aware distributed deep learning framework for heterogeneous IoT edge devices. Futur Gener Comput Syst 125:908–920
Kamal KC, Yin Z, Wu M, Wu Z (2019) Depthwise separable convolution architectures for plant disease classification. Comput Electron Agric 165:104948
Kamath V, Renuka A (2023) Deep learning based object detection for resource constrained devices: systematic review, future trends and challenges ahead. Neurocomputing 531:34–60
Kang H, Zhou H, Wang X, Chen C (2020) Real-time fruit recognition and grasping estimation for robotic apple harvesting. Sensors 20(19):5670
Ke X, Lin X, Qin L (2021) Lightweight convolutional neural network-based pedestrian detection and re-identification in multiple scenarios. Mach vis Appl 32:1–23
Kim W, Jung WS, Choi HK (2019) Lightweight driver monitoring system based on multi-task mobilenets. Sensors 19(14):3200
Kim K, Jang SJ, Park J, Lee E, Lee SS (2023) Lightweight and energy-efficient deep learning accelerator for real-time object detection on edge devices. Sensors 23(3):1185
Kim KH, Hong S, Roh B, Cheon Y, and Park M (2016) Pvanet: Deep but lightweight neural networks for real-time object detection. arXiv preprint arXiv:1608.08021.
Kondaveeti HK, Kumaravelu NK, Vanambathina SD, Mathe SE, Vappangi S (2021) A systematic literature review on prototyping with Arduino: applications, challenges, advantages, and limitations. Comput Sci Rev 40:100364
Kong T, Sun F, Liu H, Jiang Y, Li L, Shi J (2020a) Foveabox: beyound anchor-based object detection. IEEE Trans Image Process 29:7389–7398
Kong Z, Xiong F, Zhang C, Fu Z, Zhang M, Weng J, Fan M (2020b) Automated maxillofacial segmentation in panoramic dental X-ray images using an efficient encoder-decoder network. IEEE Access 8:207822–207833
Koubaa A, Ammar A, Kanhouch A, AlHabashi Y (2021) Cloud versus edge deployment strategies of real-time face recognition inference. IEEE Trans Netw Sci Eng 9(1):143–160
Krizhevsky A, Sutskever I, Hinton GE (2012) Imagenet classification with deep convolutional neural networks. Adv Neural Inf Process Syst. 25
Kyrkou C (2020) YOLOpeds: efficient real-time single-shot pedestrian detection for smart camera applications. IET Comput Vision 14(7):417–425
Kyrkou C (2021) C 3 Net: end-to-end deep learning for efficient real-time visual active camera control. J Real-Time Image Proc 18(4):1421–1433
Kyrkou C, Theocharides T (2020) EmergencyNet: efficient aerial image classification for drone-based emergency monitoring using atrous convolutional feature fusion. IEEE J Sel Top Appl Earth Observ Remote Sens 13:1687–1699
Lai CY, Wu BX, Shivanna VM, Guo JI (2021) MTSAN: multi-task semantic attention network for ADAS applications. IEEE Access 9:50700–50714
Lan H, Meng J, Hundt C, Schmidt B, Deng M, Wang X, Liu W, Qiao Y, Feng S (2019) FeatherCNN: fast inference computation with TensorGEMM on ARM architectures. IEEE Trans Parallel Distrib Syst 31(3):580–594
Law H, Deng J (2018) Cornernet: detecting objects as paired keypoints. In Proceedings of the European conference on computer vision (ECCV) (pp. 734–750)
Law H, Teng Y, Russakovsky O, Deng J (2019) Cornernet-lite: efficient keypoint based object detection. arXiv preprint arXiv:1904.08900
Li J, Ye J (2023) Edge-YOLO: lightweight infrared object detection method deployed on edge devices. Appl Sci 13(7):4402

Li X, Wang W, Wu L, Chen S, Hu X, Li J, Tang J, Yang J (2020a) Generalized focal loss: learning qualified and distributed bounding boxes for dense object detection. Adv Neural Inf Process Syst 33:21002–21012
Li P, Han L, Tao X, Zhang X, Grecos C, Plaza A, Ren P (2020b) Hashing nets for hashing: a quantized deep learning to hash framework for remote sensing image retrieval. IEEE Trans Geosci Remote Sens 58(10):7331–7345
Li Y, Li M, Qi J, Zhou D, Zou Z, Liu K (2021a) Detection of typical obstacles in orchards based on deep convolutional neural network. Comput Electron Agric 181:105932
Li Z, Liu X, Zhao Y, Liu B, Huang Z, Hong R (2021b) A lightweight multi-scale aggregated model for detecting aerial images captured by UAVs. J vis Commun Image Represent 77:103058
Li C, Fan Y, Cai X (2021c) PyConvU-Net: a lightweight and multiscale network for biomedical image segmentation. BMC Bioinf 22:1–11
Li T, Wang J, Zhang T (2022a) L-DETR: a light-weight detector for end-to-end object detection with transformers. IEEE Access 10:105685–105692
Li S, Yang Z, Nie H, Chen X (2022b) Corn disease detection based on an improved YOLOX-Tiny network model. Int J Cognit Inform Nat Intell (IJCINI) 16(1):1–8
Li H, Lin Z, Shen X, Brandt J, Hua G (2015) A convolutional neural network cascade for face detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 5325–5334)
Li Z, Peng C, Yu G, Zhang X, Deng Y, Sun J (2017) Light-head r-cnn: In defense of two-stage object detector. arXiv preprint arXiv:1711.07264
Li Y, Li J, Lin W, Li J (2018) Tiny-DSOD: lightweight object detection for resource-restricted usages. arXiv preprint arXiv:1807.11013
Li Y, Chen Y, Wang N, Zhang Z (2019) Scale-aware trident networks for object detection. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 6054–6063)
Liang L, Wang G (2021) Efficient recurrent attention network for remote sensing scene classification. IET Image Proc 15(8):1712–1721
Lin TY, Maire M, Belongie S, Hays J, Perona P, Ramanan D, Dollár P, Zitnick CL (2014) Microsoft coco: Common objects in context. In Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13 (pp. 740–755). Springer International Publishing
Lin TY, Goyal P, Girshick R, He K, Dollár P (2017) Focal loss for dense object detection. In Proceedings of the IEEE international conference on computer vision (pp. 2980–2988)
Lin TY, Dollár P, Girshick R, He K, Hariharan B, Belongie S (2017) Feature pyramid networks for object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 2117–2125)
Liu L, Ouyang W, Wang X, Fieguth P, Chen J, Liu X, Pietikäinen M (2020a) Deep learning for generic object detection: a survey. Int J Comput Vision 128:261–318
Liu X, Liu B, Liu G, Chen F, Xing T (2020b) Mobileaid: a fast and effective cognitive aid system on mobile devices. IEEE Access 8:101923–101933
Liu J, Li Q, Cao R, Tang W, Qiu G (2020c) MiniNet: an extremely lightweight convolutional neural network for real-time unsupervised monocular depth estimation. ISPRS J Photogramm Remote Sens 166:255–267
Liu X, Li Y, Shuang F, Gao F, Zhou X, Chen X (2020d) ISSD: improved SSD for insulator and spacer online detection based on UAV system. Sensors 20(23):6961
Liu Y, Sun P, Wergeles N, Shang Y (2021a) A survey and performance evaluation of deep learning methods for small object detection. Expert Syst Appl 172:114602
Liu S, Guo B, Ma K, Yu Z, Du J (2021b) AdaSpring: context-adaptive and runtime-evolutionary deep model compression for mobile applications. Proc ACM Interact Mobile Wearable Ubiquitous Technol 5(1):1–22
Liu Z, Ma J, Weng J, Huang F, Wu Y, Wei L, Li Y (2021c) LPPTE: a lightweight privacy-preserving trust evaluation scheme for facilitating distributed data fusion in cooperative vehicular safety applications. Inf Fus 73:144–156
Liu Y, Zhang C, Wu W, Zhang B, Zhou F (2022a) MiniYOLO: a lightweight object detection algorithm that realizes the trade-off between model size and detection accuracy. Int J Intell Syst 37(12):12135–12151
Liu T, Wang J, Huang X, Lu Y, Bao J (2022b) 3DSMDA-Net: an improved 3DCNN with separable structure and multi-dimensional attention for welding status recognition. J Manuf Syst 62:811–822
Liu S, Huang D (2018) Receptive field block net for accurate and fast object detection. In Proceedings of the European conference on computer vision (ECCV) (pp. 385–400)

Liu W, Anguelov D, Erhan D, Szegedy C, Reed S, Fu CY, Berg AC (2016) Ssd: single shot multibox detector. In Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14 (pp. 21–37). Springer International Publishing

Long F (2020) Microscopy cell nuclei segmentation with enhanced U-Net. BMC Bioinf 21(1):8

Long ZHOU, Suyuan W, Zhongma CUI, Jiaqi FANG, Xiaoting YANG, Wei D (2020b) Lira-YOLO: a lightweight model for ship detection in radar images. J Syst Eng Electron 31(5):950–956

Long X, Deng K, Wang G, Zhang Y, Dang Q, Gao Y, Shen H, Ren J, Han S, Ding E, Wen S (2020) PP-YOLO: An effective and efficient implementation of object detector. arXiv preprint arXiv:2007.12099

Lu Y, Zhang L, and Xie W (2020) YOLO-compact: an efficient YOLO network for single category real-time object detection. In 2020 Chinese control and decision conference (CCDC) (pp. 1931–1936). IEEE

Luo X, Zhu J, Yu Q (2019) Efficient convNets for fast traffic sign recognition. IET Intel Transport Syst 13(6):1011–1015

Ma N, Yu X, Peng Y, Wang S (2019) A lightweight hyperspectral image anomaly detector for real-time mission. Remote Sens 11(13):1622

Ma M, Ma W, Jiao L, Liu X, Li L, Feng Z, Yang S (2023) A multimodal hyper-fusion transformer for remote sensing image classification. Inf Fus 96:66–79

Ma N, Zhang X, Zheng HT, Sun J (2018) Shufflenet v2: practical guidelines for efficient CNN architecture design. In Proceedings of the European conference on computer vision (ECCV) (pp. 116–131)

Makantasis K, Karantzalos K, Doulamis A, Doulamis N (2015) Deep supervised learning for hyperspectral data classification through convolutional neural networks. In 2015 IEEE international geoscience and remote sensing symposium (IGARSS) (pp. 4959–4962). IEEE

Makkar A, Ghosh U, Rawat DB, Abawajy JH (2021) Fedlearnsp: preserving privacy and security using federated learning and edge computing. IEEE Consumer Electron Mag 11(2):21–27

Mansouri SS, Kanellakis C, Kominiak D, Nikolakopoulos G (2020) Deploying MAVs for autonomous navigation in dark underground mine environments. Robot Auton Syst 126:103472

Mao QC, Sun HM, Liu YB, Jia RS (2019) Mini-YOLOv3: real-time object detector for embedded applications. IEEE Access 7:133529–133538

Mehta S, Rastegari M (2021) Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. arXiv preprint arXiv:2110.02178

Mittal P, Singh R, Sharma A (2020) Deep learning-based object detection in low-altitude UAV datasets: a survey. Image vis Comput 104:104046

Muhammad K, Hussain T, Del Ser J, Palade V, De Albuquerque VHC (2019) DeepReS: a deep learning-based video summarization strategy for resource-constrained industrial surveillance scenarios. IEEE Trans Industr Inf 16(9):5938–5947

Nguyen HD, Na IS, Kim SH, Lee GS, Yang HJ, Choi JH (2019) Multiple human tracking in drone image. Multimedia Tools Appl 78:4563–4577

Nguyen TV, Tran AT, Dao NN, Moon H, Cho S (2023) Information fusion on delivery: a survey on the roles of mobile edge caching systems. Inf Fus 89:486–509

Ogden SS, Guo T (2019) Characterizing the deep neural networks inference performance of mobile applications. arXiv preprint arXiv:1909.04783

Ophoff T, Van Beeck K, Goedemé T (2019) Exploring RGB+ Depth fusion for real-time object detection. Sensors 19(4):866

Ouyang Z, Niu J, Liu Y, Guizani M (2019) Deep CNN-based real-time traffic light detector for self-driving vehicles. IEEE Trans Mob Comput 19(2):300–313

Paluru N, Dayal A, Jenssen HB, Sakinis T, Cenkeramaddi LR, Prakash J, Yalavarthy PK (2021) Anam-Net: anamorphic depth embedding-based lightweight CNN for segmentation of anomalies in COVID-19 chest CT images. IEEE Trans Neural Netw Learn Syst 32(3):932–946

Panero Martinez R, Schiopu I, Cornelis B, Munteanu A (2021) Real-time instance segmentation of traffic videos for embedded devices. Sensors 21(1):275

Pang J, Li C, Shi J, Xu Z, and Feng H (2019) R2-CNN: fast tiny object detection in large-scale remote sensing images. arXiv 2019. arXiv preprint arXiv:1902.06042

Paoletti ME, Haut JM, Pereira NS, Plaza J, Plaza A (2021) Ghostnet for hyperspectral image classification. IEEE Trans Geosci Remote Sens 59(12):10378–10393

Picron C, Tuytelaars T (2021) Trident pyramid networks: the importance of processing at the feature pyramid level for better object detection. arXiv preprint arXiv:2110.04004

Ping P, Huang C, Ding W, Liu Y, Chiyomi M, Kazuya T (2023) Distracted driving detection based on the fusion of deep learning and causal reasoning. Inf Fus 89:121–142

Qian S, Ning C, Hu Y (2021) MobileNetV3 for image classification. In 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE) (pp. 490–497). IEEE

Qin Z, Li Z, Zhang Z, Bao Y, Yu G, Peng Y, Sun J (2019) ThunderNet: towards real-time generic object detection on mobile devices. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 6718–6727)

Qin S, Liu S (2020) Efficient and unified license plate recognition via lightweight deep neural network. IET Image Proc 14(16):4102–4109

Quang TN, Lee S, Song BC (2021) Object detection using improved bi-directional feature pyramid network. Electronics 10(6):746

Ran X, Chen H, Liu Z, Chen J (2017) Delivering deep learning to mobile devices via offloading. In Proceedings of the Workshop on Virtual Reality and Augmented Reality Network (pp. 42–47)

Rani E (2021) LittleYOLO-SPP: a delicate real-time vehicle detection algorithm. Optik 225:165818

Redmon J, Farhadi A (2017) YOLO9000: better, faster, stronger. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 7263–7271)

Redmon J, Farhadi A (2018) Yolov3: an incremental improvement. arXiv preprint arXiv:1804.02767

Redmon J, Divvala S, Girshick R, Farhadi A (2016) You only look once: unified, real-time object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 779–788)

Ren S, He K, Girshick R, Sun J (2015) Faster r-cnn: towards real-time object detection with region proposal networks. Adv Neural Inf Process Syst. https://doi.org/10.1109/TPAMI.2016.2577031

Ren J, Guo Y, Zhang D, Liu Q, Zhang Y (2018) Distributed and efficient object detection in edge computing: challenges and solutions. IEEE Netw 32(6):137–143

Rodriguez-Conde I, Campos C, Fdez-Riverola F (2021) On-device object detection for more efficient and privacy-compliant visual perception in context-aware systems. Appl Sci 11(19):9173

Rui Z, Zhaokui W, Yulin Z (2019) A person-following nanosatellite for in-cabin astronaut assistance: system design and deep-learning-based astronaut visual tracking implementation. Acta Astronaut 162:121–134

Saidi A, Othman SB, Dhouibi M, Saoud SB (2021) FPGA-based implementation of classification techniques: a survey. Integration 81:280–299

Samore A, Rusci M, Lazzaro D, Melpignano P, Benini L, Morigi S (2020) BrightNet: a deep CNN for OLED-based point of care immunofluorescent diagnostic systems. IEEE Trans Instrum Meas 69(9):6766–6775

Sandler M, Howard A, Zhu M, Zhmoginov A, Chen LC (2018) Mobilenetv2: inverted residuals and linear bottlenecks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510–4520)

Sharma VK, Mir RN (2020) A comprehensive and systematic look up into deep learning based object detection techniques: a review. Comput Sci Rev 38:100301

Shi C, Wang T, Wang L (2020) Branch feature fusion convolution network for remote sensing scene classification. IEEE J Sel Top Appl Earth Observ Remote Sens 13:5194–5210

Shoeibi A, Khodatars M, Jafari M, Ghassemi N, Moridian P, Alizadehsani R, Ling SH, Khosravi A, Alinejad-Rokny H, Lam HK, Fuller-Tyszkiewicz M (2023) Diagnosis of brain diseases in fusion of neuroimaging modalities using deep learning: a review. Inf Fus 93:85–117

Silva SH, Rad P, Beebe N, Choo KKR, Umapathy M (2019) Cooperative unmanned aerial vehicles with privacy preserving deep vision for real-time object identification and tracking. J Parallel Distrib Comput 131:147–160

Song S, Jing J, Huang Y, Shi M (2021) EfficientDet for fabric defect detection based on edge computing. J Eng Fibers Fabr 16:15589250211008346

Steimle F, Wieland M, Mitschang B, Wagner S, Leymann F (2017) Extended provisioning, security and analysis techniques for the ECHO health data management system. Computing 99:183–201

Subedi P, Hao J, Kim IK, Ramaswamy L (2021) AI multi-tenancy on edge: concurrent deep learning model executions and dynamic model placements on edge devices. In 2021 IEEE 14th International Conference on Cloud Computing (CLOUD) (pp. 31–42). IEEE

Sun Y, Pan B, Fu Y (2021) Lightweight deep neural network for real-time instrument semantic segmentation in robot assisted minimally invasive surgery. IEEE Robot Autom Lett 6(2):3870–3877

Tan M, Chen B, Pang R, Vasudevan V, Sandler M, Howard A, Le QV (2019) Mnasnet: Platform-aware neural architecture search for mobile. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 2820–2828)

Tan M, Pang R, Le QV (2020) Efficientdet: scalable and efficient object detection. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 10781–10790)

Tang Q, Li J, Shi Z, Hu Y (2020) Lightdet: a lightweight and accurate object detection network. In ICASSP 2020–2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) (pp. 2243–2247). IEEE

Tang Z, Liu X, Shen G, and Yang B (2020) Penet: object detection using points estimation in aerial images. arXiv preprint arXiv:2001.08247.

Tsai WC, Lai JS, Chen KC, Shivanna V, Guo JI (2021) A lightweight motional object behavior prediction system harnessing deep learning technology for embedded adas applications. Electronics 10(6):692

Tzelepi M, Tefas A (2020) Improving the performance of lightweight CNNs for binary classification using quadratic mutual information regularization. Pattern Recogn 106:107407

Uijlings JR, Van De Sande KE, Gevers T, Smeulders AW (2013) Selective search for object recognition. Int J Comput Vision 104:154–171

Ullah A, Muhammad K, Ding W, Palade V, Haq IU, Baik SW (2021) Efficient activity recognition using lightweight CNN and DS-GRU network for surveillance applications. Appl Soft Comput 103:107102

Véstias MP, Duarte RP, de Sousa JT, Neto HC (2020) Moving deep learning to the edge. Algorithms 13(5):125

Wang RJ, Li X, Ling CX (2018) Pelee: a real-time object detection system on mobile devices. Adv Neural Inf Process Syst. https://doi.org/10.48550/arXiv.1804.06882

Wang X, Han Y, Leung VC, Niyato D, Yan X, Chen X (2020a) Convergence of edge computing and deep learning: a comprehensive survey. IEEE Commun Surv Tutor 22(2):869–904

Wang F, Xie F, Shen S, Huang L, Sun R, Le Yang J (2020c) A novel multiface recognition method with short training time and lightweight based on ABASNet and H-softmax. IEEE Access 8:175370–175384

Wang T, Wang P, Cai S, Zheng X, Ma Y, Jia W, Wang G (2021a) Mobile edge-enabled trust evaluation for the Internet of Things. Inf Fus 75:90–100

Wang J, Huang R, Guo S, Li L, Zhu M, Yang S, Jiao L (2021c) NAS-guided lightweight multiscale attention fusion network for hyperspectral image classification. IEEE Trans Geosci Remote Sens 59(10):8754–8767

Wang D, Ren J, Wang Z, Zhang Y, Shen XS (2022a) PrivStream: a privacy-preserving inference framework on IoT streaming data at the edge. Inf Fus 80:282–294

Wang G, Ding H, Li B, Nie R, Zhao Y (2022b) Trident-YOLO: improving the precision and speed of mobile device object detection. IET Image Proc 16(1):145–157

Wang Y, Wang J, Zhang W, Zhan Y, Guo S, Zheng Q, Wang X (2022c) A survey on deploying mobile deep learning applications: a systemic and technical perspective. Digit Commun Netw 8(1):1–17

Wang X, Zhao Q, Jiang P, Zheng Y, Yuan L, Yuan P (2022d) LDS-YOLO: a lightweight small object detection method for dead trees from shelter forest. Comput Electron Agric 198:107035

Wang C, Wang Z, Li K, Gao R, Yan L (2023b) Lightweight object detection model fused with feature pyramid. Multimedia Tools Appl 82(1):601–618

Wang K, Liew JH, Zou Y, Zhou D, Feng J (2019) Panet: few-shot image semantic segmentation with prototype alignment. In Proceedings of the IEEE/CVF International Conference on Computer Vision (pp. 9197–9206)

Wang CY, Liao HYM, Wu YH, Chen PY, Hsieh JW, Yeh IH (2020) CSPNet: a new backbone that can enhance learning capability of CNN. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops (pp. 390–391).

Wang CY, Bochkovskiy A, Liao HYM (2021) Scaled-yolov4: scaling cross stage partial network. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 13029–13038)

Wang CY, Bochkovskiy A, Liao HYM (2023) YOLOv7: trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 7464–7475)

Wu Q, Wang H, Liu Y, Zhang L, Gao X (2019) SAT: single-shot adversarial tracker. IEEE Trans Industr Electron 67(11):9882–9892

Wu X, Sahoo D, Hoi SC (2020) Recent advances in deep learning for object detection. Neurocomputing 396:39–64

Wu Y, Feng S, Huang X, Wu Z (2021) L4Net: an anchor-free generic object detector with attention mechanism for autonomous driving. IET Comput Vision 15(1):36–46

Xiao Y, Tian Z, Yu J, Zhang Y, Liu S, Du S, Lan X (2020) A review of object detection based on deep learning. Multimedia Tools Appl 79:23729–23791

Xu D, Wu Y (2021) FE-YOLO: a feature enhancement network for remote sensing target detection. Remote Sens 13(7):1311

Xu Z, Liu W, Huang J, Yang C, Lu J, Tan H (2020) Artificial intelligence for securing IoT services in edge computing: a survey. Secur Commun Netw 2020(1):8872586
Xu C, Zhu G, Shu J (2021) A lightweight and robust lie group-convolutional neural networks joint representation for remote sensing scene classification. IEEE Trans Geosci Remote Sens 60:1–15
Xu M, Liu J, Liu Y, Lin F X, Liu Y, Liu X (2019) A first look at deep learning apps on smartphones. In The World Wide Web Conference (pp. 2125–2136)
Xu S, Wang X, Lv W, Chang Q, Cui C, Deng K, Wang G, Dang Q, Wei S, Du Y, Lai B (2022) PP-YOLOE: an evolved version of YOLO. arXiv preprint arXiv:2203.16250
Yang Z, Rothkrantz, L (2011) Surveillance system using abandoned object detection. In Proceedings of the 12th international conference on computer systems and technologies (pp. 380–386)
Yang Z, Liu S, Hu H, Wang L, Lin S (2019) Reppoints: point set representation for object detection. In Proceedings of the IEEE/CVF international conference on computer vision (pp. 9657–9666)
Yi Z, Yongliang S, Jun Z (2019) An improved tiny-yolov3 pedestrian detection algorithm. Optik 183:17–23
Yin R, Zhao W, Fan X, Yin Y (2020) AF-SSD: an accurate and fast single shot detector for high spatial remote sensing imagery. Sensors 20(22):6530
Yin T, Chen W, Liu B, Li C, Du L (2023) Light "You Only Look Once": an improved lightweight vehicle-detection model for intelligent vehicles under dark conditions. Mathematics 12(1):124
Yu J, Jiang Y, Wang Z, Cao Z, Huang T (2016) Unitbox: an advanced object detection network. In Proceedings of the 24th ACM International Conference on Multimedia (pp. 516–520)
Yu G, Chang Q, Lv W, Xu C, Cui C, Ji W, Dang Q, Deng K, Wang G, Du Y, Lai B, Ma Y (2021) PP-PicoDet: a better real-time object detector on mobile devices. arXiv preprint arXiv:2111.00902
Yuan F, Zhang L, Wan B, Xia X, Shi J (2019) Convolutional neural networks based on multi-scale additive merging layers for visual smoke recognition. Mach vis Appl 30:345–358
Zaidi S, Ansari SA, Aslam MS, Kanwal N, Asghar M, Lee B (2022) A survey of modern deep learning based object detection models. Digit Sig Process 126:103514
Zhang S, Wang X, Lei Z, Li SZ (2019a) Faceboxes: a CPU real-time and accurate unconstrained face detector. Neurocomputing 364:297–309
Zhang Y, Liu M, Chen Y, Zhang H, Guo Y (2019b) Real-time vision-based system of fault detection for freight trains. IEEE Trans Instrum Meas 69(7):5274–5284
Zhang X, Lin X, Zhang Z, Dong L, Sun X, Sun D, Yuan K (2020b) Artificial intelligence medical ultrasound equipment: application of breast lesions detection. Ultrason Imaging 42(4–5):191–202
Zhang S, Li Y, Liu X, Guo S, Wang W, Wang J, Ding B, Wu D (2020c) Towards real-time cooperative deep inference over the cloud and edge end devices. Proc ACM Interact Mobile Wearable Ubiquitous Technol 4(2):1–24
Zhang Y, Zhang H, Huang Q, Han Y, Zhao M (2024) DsP-YOLO: an anchor-free network with DsPAN for small object detection of multiscale defects. Expert Syst Appl 241:122669
Zhang S, Wen L, Bian X, Lei Z, Li SZ (2018) Single-shot refinement neural network for object detection. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4203–4212)
Zhang X, Zhou X, Lin M, Sun J (2018) Shufflenet: an extremely efficient convolutional neural network for mobile devices. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 6848–6856)
Zhang S, Chi C, Yao Y, Lei Z, Li SZ (2020) Bridging the gap between anchor-based and anchor-free detection via adaptive training sample selection. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 9759–9768)
Zhao ZQ, Zheng P, Xu ST, Wu X (2019) Object detection with deep learning: a review. IEEE Trans Neural Netw Learn Syst 30(11):3212–3232
Zhao H, Zhou Y, Zhang L, Peng Y, Hu X, Peng H, Cai X (2020a) Mixed YOLOv3-LITE: a lightweight real-time object detection method. Sensors 20(7):1861
Zhao Z, Zhang Z, Xu X, Xu Y, Yan H, Zhang L (2020b) A lightweight object detection network for real-time detection of driver handheld call on embedded devices. Comput Intell Neurosci 2020(1):6616584
Zhao Y, Yin Y, Gui G (2020c) Lightweight deep learning based intelligent edge surveillance techniques. IEEE Trans Cognit Commun Netw 6(4):1146–1154
Zheng G, Chai WK, Duanmu JL, Katos V (2023) Hybrid deep learning models for traffic prediction in large-scale road networks. Inf Fus 92:93–114
Zhou Y (2024) A YOLO-NL object detector for real-time detection. Expert Syst Appl 238:122256
Zhou T, Fan DP, Cheng MM, Shen J, Shao L (2021a) RGB-D salient object detection: a survey. Comput Visual Media 7:37–69

Zhou X, Li X, Hu K, Zhang Y, Chen Z, Gao X (2021b) ERV-Net: an efficient 3D residual neural network for brain tumor segmentation. Expert Syst Appl 170:114566

Zhou L, Rao X, Li Y, Zuo X, Qiao B, Lin Y (2022) A lightweight object detection method in aerial images based on dense feature fusion path aggregation network. ISPRS Int J Geo Inf 11(3):189

Zhou X, Wang D, Krähenbühl P (2019) Objects as points. arXiv preprint arXiv:1904.07850

Zhou X, Zhuo J, Krahenbuhl P (2019) Bottom-up object detection by grouping extreme and center points. In Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (pp. 850–859)

Zhou L, Wei S, Cui Z, Ding W (2019) YOLO-RD: a lightweight object detection network for range doppler radar images. In IOP Conference Series: Materials Science and Engineering (Vol. 563, No. 4, p. 042027). IOP Publishing

Zhu Z, He X, Qi G, Li Y, Cong B, Liu Y (2023) Brain tumor segmentation based on the fusion of deep semantics and edge information in multimodal MRI. Inf Fus 91:376–387

Zitnick CL, Dollár P (2014) Edge boxes: locating object proposals from edges. In Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6–12, 2014, Proceedings, Part V 13 (pp. 391–405). Springer International Publishing

Zou Z, Chen K, Shi Z, Guo Y, Ye J (2023) Object detection in 20 years: a survey. Proc IEEE 111(3):257–276

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.